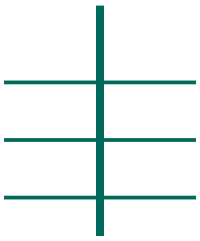
MEDPHARM ERP

Service Manual

Operations, Maintenance, and Recovery Handbook



Volume II — Operator Edition

Revision 1.7.5-A // Issued April 2026

Enlightec Ltd. · Medical & Pharmaceutical Enterprise Resource Planning

Field	Value
Title	MedPharm ERP — Service Manual
Volume / Edition	Volume II — Operator Edition
Revision	1.7.5-A
Issue Date	25 April 2026
Author	Robert Andrew Stillwell
Publisher	Enlightec Ltd., www.enlightec.com
Applies To	MedPharm ERP versions 1.7.x (latest 1.7.5)
Supersedes	No previous edition
Classification	CONFIDENTIAL — Authorised Operators
Licence	GNU General Public License v3.0 (content redistributable)
Companion Volume	Volume I — Technical Reference (MedPharm_ERP_Documentation.pdf)

Copyright and Attribution

MedPharm ERP is software, not a medical device, and is not intended to diagnose, treat, cure, or prevent any disease. Clinical decisions must remain the responsibility of a licensed practitioner. The drug-drug interaction data shipped with the seeded database is illustrative; operators deploying MedPharm in a regulated clinical setting must supply and validate a current, authoritative interaction dataset.

How to Contact the Publisher

General correspondence should be addressed to Enlightec Ltd. via the author, Robert Andrew Stillwell, at **Andrew.Stillwell@enlightec.com**. Issues with the software itself are best reported through the project's source tracker so that they may be triaged alongside code contributions. Suggestions for improvement of this manual — corrections, clarifications, missing procedures — are actively welcomed and should carry the subject line *Service Manual Feedback*.

Table of Contents

Part VI — Maintenance and Change

Part VII — Fault Diagnosis

Part VIII — Business Continuity

Part IX — Appendices

Foreword

Software that touches health records has an uncommonly long shadow. A patient who never met the engineers who built the system will, even so, feel their decisions in small and personal ways — in how long the prescription queue takes at a pharmacy counter, in whether an allergy note is visible at the right moment, in whether the portal loads when they need it most. MedPharm ERP has been written with that weight in mind, and this manual is written to the people who carry that weight after the code is compiled: the operators.

A service manual is a quiet companion. It sits on the shelf — physical or digital — until the day something unusual happens, and then it is expected to speak plainly, without drama, about what to do. The temptation when writing such a manual is to reduce every procedure to a list of bare imperatives. That temptation is worth resisting. A list of commands is useful only to a reader who already understands the system; a reader under pressure, perhaps new to the platform and working outside business hours, is better served by prose that explains the *shape* of the problem before prescribing a response. We have therefore tried, throughout, to describe systems and intentions first and commands second.

MedPharm ERP is a deliberately modest piece of software. It runs on ordinary Linux hosts, it stores its data in a single SQLite file, and it speaks a small, well-understood REST dialect to its mobile and desktop clients. That modesty is a design position: the more moving parts a system has, the more opportunities it offers for misconfiguration, and the more difficult it becomes to reason about during an incident. Operators should find that MedPharm rewards careful attention with long stretches of uneventful service, punctuated by upgrades that go cleanly and backups that restore on the first try. Those outcomes are not automatic; they are the product of the disciplines described in the pages that follow.

This volume is the operator's counterpart to the MedPharm ERP Technical Reference (Volume I). Where Volume I describes what the software *is*, Volume II describes what it *does under your care*. The two are designed to be used together; where procedures here require a fuller understanding of the underlying code, the relevant chapter of Volume I is cited by name.

Finally, a note on language. We have written this manual in the second person ("you", "the operator") because that is the voice in which operational work is actually done. The passive voice has its place in specifications; it has none in a document whose purpose is to help a person make a correct decision while a phone is ringing. If you find a passage here that seems vague, or that hedges where it should commit, please report it. A service manual is never finished — it is only current.

Robert Andrew Stillwell
Enlightec Ltd.

How to Use This Manual

This manual has been written to be read in three quite different ways, and it rewards each of them differently.

Reading cover-to-cover

If you are new to the MedPharm platform — a new hire inheriting the service, perhaps, or a consultant engaged to assess a deployment — read this manual from Part I through to Part VIII in order. Each chapter builds on the previous one without heavily repeating it, and the shape of the system emerges cleanly this way. A motivated reader will complete a cover-to-cover pass in a long afternoon; a second pass, performed once you have had a week or two of hands-on operation, will be far shorter and far more useful than the first.

As a reference

Most of the manual's working life will be spent as a reference volume, consulted briefly to answer a specific question. The Table of Contents is granular enough for this purpose; individual chapters are self-contained and open with a short statement of scope. Where a procedure depends on prior state (for example, a restore procedure that assumes a healthy backup is on hand), that prior state is named explicitly. Consequently, you should not need to read more than one chapter at a time to answer most questions.

Under incident pressure

The third — and hopefully rarest — mode is consultation during an active incident. Part VII (Fault Diagnosis) and Part VIII (Business Continuity) have been written specifically to be usable under pressure. Symptoms are indexed by what the operator can actually observe at the time (a failing request, a suspicious log line, a stalled container) rather than by what they might eventually turn out to mean. Decision points within runbooks are explicit: a runbook that cannot be safely followed without an additional check will say so, and tell you which check to run.

A word on completeness

No manual can anticipate every situation. We have therefore taken two measures to keep this document honest. First, procedures describe *what* must be accomplished, not only *how* we currently accomplish it; that separation makes the manual resilient to small changes in tooling. Second, each major procedure ends with a verification step — a test that distinguishes "the command ran" from "the outcome was achieved". Where you find yourself working around the manual rather than with it, treat that as a bug report and file it accordingly.

Conventions and Signal Words

Consistent typographic and editorial conventions are used throughout this manual so that operators may scan a page and judge its relevance before committing to a full read.

Typography

Commands, filenames, environment variable names, and HTTP routes appear in a monospaced font, for example `./install.sh --docker`, `/etc/ssl/medpharm/fullchain.pem`, `MEDPHARM_JWT_SECRET`, or `POST /api/v1/auth/login/patient`. Longer command-line examples appear in set-apart code blocks, which reproduce the content faithfully including line breaks but not terminal colour.

Signal Words

Four signal-word boxes are used throughout the manual. Their meanings are precise and ought not be mixed.

Note. A Note conveys information that is useful but not strictly necessary to carry out the procedure. Skipping a Note will not cause harm; reading one will often save time.

Caution. A Caution introduces information that, if ignored, may cause degraded service, confusing behaviour, or subsequent extra work. No permanent damage is expected.

Danger. A Danger introduces information that, if ignored, may cause data loss, security compromise, or service unavailability. An operator should not proceed past a Danger without understanding why it applies.

Legend. A Legend explains symbols, states, or enumerations used nearby — for example, the meaning of the various claim-processing statuses, or the fields shown in a particular table.

Verbs of Requirement

This manual follows the common engineering convention under which **must** indicates a firm requirement, **should** indicates a strong recommendation whose violation must be justified in writing, and **may** indicates a genuinely permissive option. **Must not** and **should not** mirror their positive forms.

Time and Date Notation

All timestamps cited in examples and logs use ISO 8601 with a timezone offset. Operators should configure MedPharm hosts to log in UTC regardless of the human operator's local timezone; this convention simplifies incident correlation across a geographically distributed team and is quietly assumed by most of the chapters that follow.

PART I

MedPharm ERP Service Manual

Orientation

Who we are, what we run, and who is responsible for it.

CHAPTER 1

The MedPharm ERP Platform at a Glance

MedPharm ERP is an enterprise resource planning system for small-to-medium medical practices and retail pharmacies. It unifies, under a single relational database, the operational concerns that such a business typically spreads across three or four uncoordinated applications: patient demographics and clinical records, prescription writing with drug-drug interaction checking, appointment scheduling, invoicing and payment capture, insurance-claim submission and adjudication, and a reference library of medications, symptoms, and diagnosed conditions. A handful of supplementary capabilities — audit logging, role-based access control, and a set of analytics dashboards — round the platform out into a serviceable workhorse rather than a mere point solution.

From an operator's vantage, the MedPharm deployment is best thought of as a constellation of six user-facing surfaces that all attach to a single, shared backend. The backend consists of a SQLite database and the SQLAlchemy ORM layer (together, the *DatabaseManager*), plus a Flask REST API that presents a JWT-authenticated HTTP interface on port 8080. Five of the six user-facing surfaces speak to the REST API over the network; the sixth — the PyQt6 desktop application for clinical staff — speaks to the DatabaseManager directly, which is appropriate only when it runs on the same host as the database file.

The six user-facing surfaces

The surfaces differ in audience, runtime, and authentication flow, but they offer substantially overlapping clinical and administrative functionality. Operators who have learned one will find the others largely self-explanatory.

Surface	Audience	Runtime	Network Dependence
PyQt6 Desktop	Clinical staff	Python 3.10+ on Linux / macOS / Windows	None (reads SQLite directly)
Flask Web Portal	Patients	Gunicorn behind Nginx, port 5000	Same host or reverse-proxied
Cloud REST API	Mobile/desktop clients	Gunicorn with gevent, port 8080	TLS over WAN (JWT)
Android Kotlin app	Patients on Android 8.0+	ART / Retrofit / OkHttp	API over TLS
iOS SwiftUI app	Patients on iOS 16+	Native URLSession async/await	API over TLS
macOS SwiftUI app	Patients, staff on macOS 13+	Native URLSession async/await	API over TLS
Windows WPF app	Patients on Windows 10/11	.NET 8, HttpClient, DPAPI	API over TLS

Clinicians overwhelmingly prefer the PyQt desktop for its speed and latency characteristics. Patients overwhelmingly prefer the native mobile clients. The Flask portal is an essential accessibility fallback and a common onboarding path.

The backend

Underneath the surfaces lies a small, conservative stack. The database engine is SQLite, which offers an attractive combination of operational simplicity (one file, no separate daemon), low resource usage, and — critically for a system that stores protected health information — the absence of network attack surface at the storage layer. SQLite's limitations are real and become visible at higher concurrency; see Chapter 5 for the sizing envelope within which SQLite comfortably serves, and Chapter 18 for the tuning parameters that extend that envelope.

The ORM is SQLAlchemy 2.0, configured in the modern `Mapped[...]` style. Twenty-three models represent the complete domain: Patient, Provider, Appointment, Prescription, PrescriptionLine, Medication, InteractionPair, Symptom, Condition, Invoice, Payment, InsuranceClaim, Allergy, MedicalRecord, AuditLog, and approximately eight more that exist chiefly as supporting associations. Eighteen Python enums model the system's finite state machines: prescription status, appointment status, claim status, user roles, and so on. Operators are not expected to interact with the ORM directly during normal service, but an awareness of its shape is useful when reading log messages or tracing a fault.

The Flask application factory, declared in `api/app.py` and composed from `api/routes.py` and `api/auth.py`, is deliberately plain. It exposes its routes under the prefix `/api/v1/` so that future evolutions of the protocol may cohabit with older clients. Authentication is performed by an HMAC-SHA256-signed JWT; refresh tokens are supported. CORS policy is derived from the `MEDPHARM_CORS_ORIGINS` environment variable. When TLS is terminated at Nginx, the Flask layer is oblivious to it, which is the usual pattern and the easiest to reason about during incidents.

A note on scale

MedPharm ERP is architected for the single-practice and single-pharmacy case. A site with, say, three doctors, twelve exam rooms, two thousand active patients, and a hundred prescriptions per business day will be very well served by a single MedPharm deployment on a modest Linux virtual machine. Operators contemplating multi-site or multi-tenant deployments should consult Part V of the Technical Reference for the (important) architectural caveats — multi-tenancy is not provided out of the box, and attempting to retrofit it by running several SQLite databases against a single Flask process is not supported and not safe.

> **Caution.** The platform's default installation is intentionally oriented toward developer convenience. Before serving real patient data, operators must traverse the hardening checklist in Chapter 13 and rotate every shipped default credential. A MedPharm host that has been *commissioned* differs considerably from one that has merely been *installed*.

CHAPTER 2

Operator Roles and Responsibilities

Operating MedPharm ERP well is a shared endeavour. No single role within a small clinical organisation owns all of the responsibilities catalogued below; in larger organisations, each row may be filled by a team. What matters is that every responsibility has an owner, and that the owner knows they are the owner. A system that nominally has backups but whose owner would have to look up where those backups live is a system that does not, in practice, have backups.

This chapter defines the roles assumed by the rest of the manual. Where a procedure requires a specific role, the chapter in question will say so. Roles are not positions; they are hats. One person may wear several.

The System Operator

The **System Operator** is the primary audience of this manual. They are the person — or the small group of people — responsible for the continuing health of the MedPharm deployment: keeping the service running, the logs healthy, the backups rotating, the certificates current, and the upgrades applied. Where this manual says *you*, it is addressing the System Operator unless otherwise specified.

System Operators must be comfortable at a Linux shell, must understand the UNIX file permission model, and must be prepared to read log output with care. Formal DBA credentials are not required, but the operator must be able to read simple SQL and to discern a benign query from one that is about to corrupt the database. Chapter 18 contains the practical SQL that the operator is most likely to issue.

The Security Officer

The **Security Officer** owns the cryptographic posture of the deployment. This includes TLS certificate management (Chapter 14), JWT and Flask-session secret rotation (Chapter 15), access-control policy governing who may hold which staff role, and the audit-log review cadence (Chapter 11). Under HIPAA, a designated Security Officer is a regulatory requirement, not a nicety; § 164.308(a)(2) of the Security Rule makes this explicit. Where the System Operator and the Security Officer are the same person, care must be taken to preserve the separation of concerns on paper, because audit-log review performed by the operator whose actions are being audited is not an effective control.

The Privacy Officer

The **Privacy Officer** is the regulatory counterpart to the Security Officer and is also mandated by HIPAA. They are the point of contact for patient requests to access, amend, or restrict their protected health information, and they own the breach-notification workflow described in Chapter 17. From the operator's

point of view, the Privacy Officer issues requests ("please disclose the audit log for patient ID 1234 for the week of…") and receives completed work.

The Clinical Administrator

The **Clinical Administrator** — sometimes the office manager, sometimes a senior nurse, occasionally the practice owner — manages day-to-day use of the desktop application by clinical staff. Their service concerns are few but important: onboarding and offboarding of staff accounts, assignment of roles to those accounts, and the maintenance of the medication formulary in line with the practice's prescribing habits. A well-briefed Clinical Administrator takes real load off the System Operator.

The End Users

Although end users — physicians, pharmacists, patients — are not *operators*, their day-to-day experience of the system is the single most important signal the operator possesses. A user who reports that "the portal is slow today" is telling you something the health check does not. Treat such reports as telemetry; triage them with the same seriousness you would give to a spike in a Prometheus dashboard.

> **Note.** In very small practices it is common for one person to hold every role in this chapter. That is permissible and often unavoidable. It does, however, mean that the documentation of procedures becomes the principal control — the substitute for the second pair of eyes. Where you are a staff of one, invest deliberately in the runbooks in Part VII; they are what will save you on the day you are ill.

The Escalation Matrix

Every role defined in this chapter must have an identified deputy — a person who is fluent in the role and available when the primary is not. For a small practice, the deputy may be an external consultant on a retainer. For a larger one, the deputy is an understudy being groomed for the primary position. The escalation matrix — who is called, in what order, during an incident — is maintained as an appendix to the organisation's incident response plan and reproduced briefly in Chapter 17.

CHAPTER 3

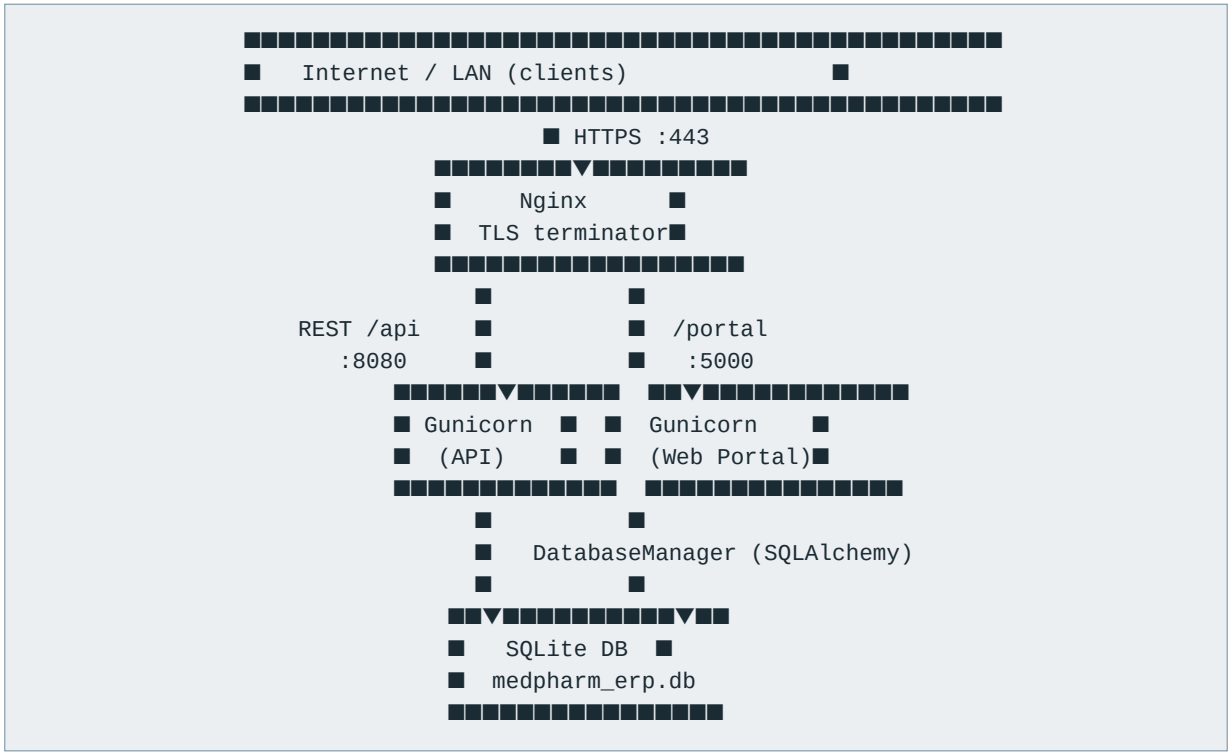
Service Topology and Control Planes

To run MedPharm ERP confidently the operator needs a firm mental picture of what is actually running, where it is running, and which process speaks to which other process. This chapter draws that picture. It is worth committing to memory; most incident triage benefits more from a clear topology than from detailed knowledge of any individual component.

Single-host deployment

The canonical deployment — and the one produced by `./install.sh --docker-server` on a clean Ubuntu 24.04 LTS host — consists of a single container running Nginx, Gunicorn (twice: once for the REST API and once for the web portal), and Supervisor as the process manager. Nginx terminates TLS on port 443 and reverse-proxies to the two Gunicorn instances. All three processes share access to the SQLite database file at `/data/medpharm_erp.db` via a named Docker volume, `medpharm-data`.

Every client — whether an Android phone across the network, a Windows desktop on the LAN, or a SwiftUI client on the nurse's iPad — enters the deployment at Nginx. The PyQt desktop is the sole exception: it may run on the same host and open the database file directly, bypassing the API entirely. This is legitimate for single-physician practices and for air-gapped backroom terminals; it is not appropriate over the network.



Two-host deployment

Once traffic exceeds the single-host envelope (see Chapter 5) the usual next step is to split Nginx onto a small edge host and keep the backend on a larger one. The two hosts communicate over a private subnet on port 8080 (plain HTTP) and 5000 (plain HTTP), and operators must enforce that the backend is not reachable from outside the private subnet. The Kubernetes manifests in `k8s/` implement this pattern and are recommended as the path of least resistance for the two-host case.

Control planes

A mature deployment has three *control planes*: independent channels by which operators exercise authority over the running system. Conflating them is a common source of outages.

Plane	Purpose	Typical Tool	Risk if Lost
Host	SSH into the Linux host	OpenSSH, bastion	Catastrophic — no recovery without it
Orchestrator	Start, stop, inspect containers	docker, docker compose, kubectl	High — service manipulation becomes manual
Application	Act inside MedPharm as admin	Desktop login as admin, API admin endpoints	Medium — clinical workflow stalls

All three planes authenticate independently. Losing one should not compromise the other two.

Data flow during a typical patient request

Consider what happens when a patient opens the iOS app and pulls down to refresh the Prescriptions list. The sequence is worth internalising because four of the most common client-side faults are simply this sequence breaking at different points.

1. The iOS client reads the JWT access token from the iOS Keychain.
2. It issues a GET against `/api/v1/patient/prescriptions` with an `Authorization: Bearer <jwt>` header.
3. Nginx terminates TLS, validates the request, and forwards it to Gunicorn on `:8080`.
4. Flask's auth middleware verifies the JWT signature using `MEDPHARM_JWT_SECRET`, extracts the patient ID, and passes the request to the route handler.
5. The route handler calls `DatabaseManager.get_prescriptions_for_patient(id)`.
6. SQLAlchemy issues a SELECT against SQLite, which returns a rowset from the file on disk.
7. The ORM materialises the rows into Prescription instances. The handler serialises them to JSON.
8. The response travels back up the stack: Gunicorn → Nginx → internet → iOS URLSession → SwiftUI View.

Legend. When a patient reports "it's stuck on loading," the fault is almost certainly at step 2, 3, or 4. When they report "it loads but shows nothing," the fault is usually at step 5 or 6. When the screen loads old data, the fault is in the client's cache, not in MedPharm at all.

PART II

MedPharm ERP Service Manual

Commissioning

The formal passage from installed to ready-to-carry-workload.

CHAPTER 4

Pre-Installation Survey

The decisions made in the hour before MedPharm ERP is installed often determine whether the deployment is serviceable for its first five years or contentious for all of them. This chapter walks through the survey that precedes a competent installation. It is designed to be used as a checklist during a pre-installation meeting with the practice's leadership; fields on the checklist that cannot be answered in the meeting should be treated as blockers.

Organisational questions

- What legal entity will own the deployment? (This is the entity whose name appears on the Business Associate Agreement and to whom audit requests are addressed.)
- How many clinical staff will use the desktop application concurrently during peak hours?
- How many patients are expected to be served in the first year? The first three years?
- Who are the named Security Officer and Privacy Officer? What are their pager rotations?
- Does the practice accept insurance, and if so, from which carriers? (This determines the initial InsuranceProvider seed.)
- What is the site's approach to patient portal identity verification? (The four-factor verifier — name, date of birth, SSN-last-4, insurance ID — is a floor, not a ceiling.)
- Who signs off on a change request before it is deployed to production?

Technical questions

- Is the host Linux, and is it Ubuntu 22.04 LTS or 24.04 LTS? (These are the supported baselines.)
- Is the host a bare VM, a Kubernetes pod, or a managed container service? (The Kubernetes overlays cover GKE, EKS, and generic ingress-nginx clusters; bare VMs are installed with install.sh.)
- Does the host have persistent storage suitable for a SQLite database that will accumulate at roughly 500 MB per 10,000 active patients?
- Will TLS certificates be issued by an internal CA, a public CA, or generated self-signed on first boot?
- What is the authoritative public DNS name for the deployment? It must match the SAN on the TLS certificate.
- What is the backup target? Object storage (S3, GCS, Linode Object Storage), local NAS, or encrypted external media?
- Is outbound traffic from the host restricted, and if so, will the backup job be able to reach the backup target?

- What observability exists already at the site? Prometheus, CloudWatch, a managed log aggregator? MedPharm's logs will need to flow somewhere.

Regulatory questions

If the deployment will store protected health information — and nearly every MedPharm deployment will — the pre-installation survey must confirm that the practice has executed a Business Associate Agreement with every vendor who may handle that information in the course of serving the application. This includes the cloud provider if one is used, the backup target if it is external, and any managed service (for example, a TLS-certificate automation service) that sits in the data path.

> **Danger.** Do not under any circumstances install MedPharm onto a host whose backup target has not signed a BAA, unless the backups are encrypted with keys that the backup provider does not possess. Encrypted-at-rest backups using provider-managed keys are insufficient; the provider must be precluded from reading the data.

Commissioning artefacts

The pre-installation survey concludes with the preparation of three artefacts that the commissioning engineer will rely on: the **Commissioning Record**, a one-page document that records every secret generated during installation (by fingerprint, never in clear); the **Topology Diagram**, a freehand sketch of every network hop between client and database; and the **Runbook Directory**, a named folder (physical or digital) in which the runbooks from Part VII are assembled for local use. These three artefacts are the inheritance that the commissioning engineer leaves for the System Operator.

CHAPTER 5

Sizing, Capacity, and Environment Planning

MedPharm ERP is a modest consumer of hardware at the scales for which it is designed, but "modest" is not "negligible," and mis-sizing it tends to manifest as intermittent slowness rather than outright failure — the worst kind of fault to diagnose. This chapter lays out the sizing envelope, the signals that suggest the envelope is being exceeded, and the scaling paths that preserve operational simplicity as demand grows.

The single-host envelope

The single-host envelope is the range of demand within which a single VM running the full-stack container (`enlightec/medpharm-server:latest`) is an unambiguously appropriate deployment. The envelope is generous and covers almost every MedPharm deployment in production today.

Dimension	Floor	Comfortable	Ceiling
Active patients	100	≤ 25,000	≤ 100,000
Concurrent staff (desktop)	1	≤ 25	≤ 50
Mobile-app requests / minute	< 1	≤ 60	≤ 200
Prescriptions written / day	1	≤ 500	≤ 2,000
Database size (on disk)	few MB	≤ 2 GB	≤ 10 GB

Once two or more of these values sit above the "comfortable" column, planning for the two-host envelope is prudent.

Hardware recommendations

Operators installing on a VM should treat the following as a minimum workable specification for production use. Development and staging environments can be thinner. Storage performance matters substantially more than CPU count, because SQLite's write throughput is dominated by fsync latency.

Profile	vCPU	RAM	Disk	Typical Host
Development / staging	2	4 GB	20 GB SSD	Small cloud VM (e2-small, t3.small, Linode 2 GB)
Small practice (≤ 5,000 patients)	2	8 GB	50 GB SSD	Midrange VM (e2-standard-2, t3.medium)
Medium practice (≤ 25,000)	4	16 GB	100 GB SSD	Dedicated VM (e2-standard-4, c5.large)
Near ceiling (≤ 100,000)	8	32 GB	200 GB NVMe	Performance VM (n2-standard-8, c5.2xlarge)

The network envelope

MedPharm's client protocols are not chatty, and the aggregate bandwidth required is small. For a medium practice, a sustained 2 Mbps egress at the site is more than sufficient; peak utilisation comes not from MedPharm but from adjacent services (backup uploads, video conferencing with patients, staff email). Latency matters more than throughput: staff perceive the desktop as sluggish if the round-trip from browser to API approaches 300 ms. Where this latency must be tolerated — for geographically distributed staff — the deployment should be placed close to the majority of users, or a regional replica considered.

Certificates and DNS

Every production deployment requires a DNS A or AAAA record resolving to the edge host, and a TLS certificate whose Common Name and Subject Alternative Name match that record. For internet-facing deployments the recommended path is Let's Encrypt via the ACME HTTP-01 or DNS-01 challenge, renewed automatically. For internal deployments, an internal CA is preferable to a self-signed certificate — self-signed certificates train users to click past trust warnings, which is itself a security problem.

Hostnames that matter

Environment	Typical Hostname	Used By
Production	erp.example-clinic.com	All mobile clients, patient portal, staff desktop
Staging	erp-staging.example-clinic.com	Release-rehearsal and training environment
Development	localhost / 127.0.0.1	Engineer workstations only

Note. The *Server URL* field on the login screen of every MedPharm mobile and desktop client allows an operator to redirect the client at a non-default host. This is invaluable when rehearsing against staging and when providing temporary access during a failover drill.

CHAPTER 6

Commissioning Procedures

Commissioning is the formal passage between "installed" and "ready to carry real workload." A commissioned MedPharm deployment has had its secrets rotated, its default accounts disabled, its certificates installed, its backup target verified, and its health probes demonstrated. Until the commissioning checklist is complete, the deployment may be demonstrable but is not trustworthy.

Step 1 — Obtain the source

Clone the repository onto the target host. Commissioning from the tip of the main branch is appropriate for development; production commissioning should pin to a tagged release to ensure reproducibility.

```
git clone https://github.com/stillwell/MedPharm.git
cd MedPharm
git checkout v1.7.5    # pin to a tagged release for production
```

Step 2 — Generate secrets

Before running the installer, generate two independent cryptographic secrets: the JWT signing key and the Flask session key. Both should be at least 256 bits of entropy. The installer will accept its own defaults if these are absent, but those defaults are known values and must not appear in a production deployment.

```
export MEDPHARM_JWT_SECRET="$(openssl rand -base64 48)"
export MEDPHARM_SECRET_KEY="$(openssl rand -base64 48)"
# Store these fingerprints in the Commissioning Record:
echo "$MEDPHARM_JWT_SECRET" | sha256sum
echo "$MEDPHARM_SECRET_KEY" | sha256sum
```

Danger. Record the fingerprints of the secrets, not the secrets themselves. Secrets stored in commissioning paperwork are a common source of compromise. If you need to share a secret, share it via a dedicated secrets vault (HashiCorp Vault, AWS Secrets Manager, 1Password Business) rather than by document.

Step 3 — Run the installer

The installer detects the host's Linux distribution, installs system packages, builds a Python virtual environment, installs application dependencies, initialises the database, and seeds the reference data. For a containerised deployment, the Docker variants perform the equivalent inside a container image.

```
# Bare-host install (single Python venv + SQLite file)
./install.sh

# Docker-based install — full stack (recommended for production)
./install.sh --docker-server
```

```
# Docker-based install — API only (recommended for separate edge + API hosts)
./install.sh --docker
```

Step 4 — Install TLS certificates

With TLS handled by Nginx in the full-stack container, the commissioning engineer drops the certificate `fullchain.pem` and private key `privkey.pem` into the `medpharm-tls` Docker volume (or the `/etc/ssl/medpharm` bind mount), and sets `MEDPHARM_TLS_MODE=require` in the environment. Once the container is restarted, the auto-generated self-signed certificate will not be used and the container will refuse to start if the supplied certificate is invalid — which is the intended behaviour, since silently falling back to a self-signed certificate in production would mask a serious misconfiguration.

Step 5 — Rotate shipped default credentials

MedPharm seeds a small cast of demonstration accounts so that an operator exploring the software can log in immediately after install. These accounts must be disabled — not merely renamed, not merely assigned new passwords, but disabled — before any patient data is entered. The commissioning engineer performs this step from the PyQt desktop logged in as the real clinical administrator, after having created at least one genuine staff account with the Admin role.

Default Username	Default Password	Commissioning Action
admin	admin123	Create real admin, then disable
dr.carter	doctor123	Disable
dr.chen	doctor123	Disable
dr.brooks	doctor123	Disable
pharm.davis	pharm123	Disable
jsmith_portal	patient123	Delete (demo patient record)
mjohnson_portal	patient123	Delete
ewilliams_portal	patient123	Delete
sdavis_portal	patient123	Delete
lmartinez_portal	patient123	Delete

Step 6 — Verify the backup pipeline

No deployment is commissioned until a successful backup has been written to the production backup target, a fresh empty VM has retrieved that backup, and the deployment on the fresh VM has served a single test request. Commissioning without a verified restore path is not commissioning — it is optimistic installation.

The backup and restore procedures live in Chapters 19 and 20. The commissioning engineer should rehearse them in both directions — a full backup, followed by a full restore onto a throw-away host — and should sign the Commissioning Record only after that rehearsal concludes without manual intervention.

> **Caution.** It is tempting at commissioning time, under schedule pressure, to mark "backup configured" once the cron job is in place and defer the restore test. Resist this. A backup that has never been restored has a meaningful probability of being unrestorable for reasons the operator cannot predict in advance — wrong encryption key, wrong compression format, incomplete schema dump, silent truncation.

CHAPTER 7

Post-Commissioning Validation

Once the steps in Chapter 6 have been executed, the deployment enters a short but important validation window, during which the commissioning engineer exercises the system end-to-end and compares the observed behaviour against the specification. The output of this chapter is a signed Validation Report that the System Operator inherits along with the running system.

Health probes

The quickest sanity check is a GET against the health endpoint from a machine on the public side of the deployment. It should return 200 OK with a JSON `{"status": "ok"}` payload in less than 200 milliseconds.

```
curl -sv https://erp.example-clinic.com/api/v1/health
# Expect:
#   HTTP/2 200
#   content-type: application/json
#   {"status": "ok"}
```

The portal should return a login page on GET of `https://erp.example-clinic.com/portal/` and the response should contain the string "MedPharm" in the HTML body. Any deviation from this is a commissioning defect and must be resolved before the deployment is handed over.

Functional validation matrix

The matrix below enumerates the minimum functional checks that every commissioning run must pass. It is reproduced as a checklist in Appendix F of this manual and is the single most-reused artefact of the commissioning process.

#	Check	Expected Outcome
1	Log in to PyQt desktop as real Admin account	Dashboard loads; KPIs render
2	Register a synthetic test patient via portal	Four-factor verifier accepts; username created
3	Prescribe medication to synthetic patient via desktop	Prescription saves; invoice auto-generated
4	Log in to iOS app as synthetic patient	Dashboard loads; prescription appears
5	Submit insurance claim from desktop billing module	Claim moves to SUBMITTED status
6	Process claim to APPROVED / PAID	Payment auto-created; invoice reflects paid balance

#	Check	Expected Outcome
7	Trigger drug-interaction warning with two known pairs	Warning dialog appears with severity
8	Restart container and verify data persists	Synthetic patient still present after restart
9	Execute backup, restore onto fresh host, log in	Data present on restored host
10	Rotate TLS certificate; verify no downtime	Clients continue to connect without retry loops

Sign-off

The Validation Report is signed by the commissioning engineer, countersigned by the System Operator taking custody, and retained on the practice's internal compliance drive for the deployment's lifetime. A deployment without a signed Validation Report is, for the purpose of this manual, not in production — it is merely in use. That distinction matters because it determines whether operator interventions should be logged as routine or as emergency.

PART III

MedPharm ERP Service Manual

Daily Operations

The quiet disciplines that produce long stretches of uneventful service.

CHAPTER 8

The Daily Operational Routine

The operational difference between a well-run MedPharm deployment and a poorly-run one is not dramatic at any single instant; it accumulates slowly out of the disciplines practiced every working day. This chapter describes a daily routine that an operator can complete in about fifteen minutes. It is not the only such routine that would suffice — organisations should feel free to adapt it — but it is a complete routine, and it is a useful reference point for improvements.

Morning: the operator's walk-around

Each business morning — ideally before staff arrive, so that any necessary intervention happens before the day's clinical work — the operator performs a short sequence that the industry calls, evocatively, a walk-around. The sequence is borrowed from the aviation ground-crew checklist: it looks at the handful of signals that most often reveal a quiet overnight problem.

1. **Health endpoint.** Curl /api/v1/health from a client on the public side. Expect HTTP 200 in under 200 ms.
2. **Container status.** Run `docker ps` (or `kubectl get pods -n medpharm`) and confirm every container is *Up* and not in a restart loop.
3. **Disk space.** Run `df -h /data` and confirm at least 20% free on the database volume.
4. **Backup proof.** Confirm that the most recent backup file exists at the expected path and was written within the last 24 hours.
5. **Audit-log tail.** Open the last 20 AuditLog rows and confirm they look typical. Unexpected admin-role events are the single most valuable early-warning signal MedPharm emits.
6. **Certificate expiry.** Check `openssl x509 -enddate` against the deployed certificate once per week. Set a calendar reminder 30 days before expiry.

If any of the six checks fails, the operator's day begins with the corresponding chapter of Part VII rather than with clinical work. Where all six pass, the operator moves on to the weekly rotation of longer-form tasks described below.

Midday: the user-facing pulse

At around midday, ideally after a morning clinic has concluded its first few hours of work, the operator spends a few minutes with the end users themselves. This is not a technical check — it is a social one. Ask the front-desk staff whether the desktop is behaving. Ask a pharmacist whether interaction warnings are appearing at expected latencies. If staff mention a glitch they have not reported formally, treat it as telemetry and follow up. The goal is to find problems before they compound, which they do with surprising speed once they enter the clinical workflow.

Evening: the close-of-day check

After the last patient of the day is seen, a short close-of-day sequence captures the day's state for recovery purposes.

1. Confirm the nightly backup cron is scheduled and has not been disabled or mis-edited.
2. Review any PENDING or PARTIAL invoices with outstanding balance older than 14 days — this is a billing concern, not a technical one, but it often reveals a technical cause.
3. Check the InsuranceClaim queue for claims stuck in IN_REVIEW longer than their service-level expectation.
4. Review the day's AuditLog entries categorised as *LOGIN_FAILED* to distinguish typos from probable probes.

The weekly rotation

Seven tasks, one per day of the week, complete the weekly rotation. None of them takes long; together they ensure that nothing important goes un-inspected for more than a week.

Day	Task	Output
Monday	Review full week's audit log for anomalies	Short note in the operator's journal
Tuesday	Rotate application log files if not automated	Previous week's logs archived / compressed
Wednesday	Restore most recent backup onto throw-away host	Confirmed restorability
Thursday	Check upstream package security advisories	List of packages requiring update
Friday	Apply any pending non-urgent security updates	Updated image; deployment unchanged
Saturday	Verify TLS cert remaining days; rotate if < 30	Healthy certificate
Sunday	Rest, on call only	Recovery from the rest of the week

Note. The point of the rotation is to prevent any single Friday afternoon from becoming "that maintenance Friday" — the one the operator dreads. Work that is distributed over a week is work that is done; work that is concentrated into a single afternoon becomes work that is deferred.

CHAPTER 9

Controlled Startup and Shutdown

Starting and stopping MedPharm ERP is, on the happy path, a single docker-compose command. This chapter describes what that command actually does, how to verify its effects, and how to perform a controlled shutdown in situations where the happy path is not available.

The controlled startup

A controlled startup initiates the supervisor process, which in turn launches Nginx, the two Gunicorn workers, and any sidecar processes. In the full-stack image, the supervisor configuration also performs one-time initialisation tasks: if the database file does not exist, it creates and seeds it; if the TLS directory is empty and the TLS mode is `auto`, it generates a self-signed certificate.

```
# Source install
./start_cloud.sh   # API server on port 8080
./start_web.sh     # Patient portal on port 5000
./start_desktop.sh # PyQt desktop (requires display server)

# Docker, full stack
cd server
docker compose -f docker-compose.hub.yml up -d

# Docker, API only
docker compose -f docker-compose.hub.yml up -d
```

Once up, verify that each component has settled. A freshly-started container is not, by virtue of having started, ready to serve traffic; the database may still be warming, the gunicorn workers may still be forking. Allow thirty seconds and then query the health endpoint.

The controlled shutdown

A controlled shutdown allows in-flight HTTP requests to complete, flushes the SQLite write-ahead log, and closes the database cleanly. It is the shutdown path to prefer except in the rare case where the process has become unresponsive.

```
# Graceful shutdown
docker compose -f docker-compose.hub.yml stop
# or
docker compose -f docker-compose.hub.yml down

# Graceful-with-timeout (30s then kill)
docker compose -f docker-compose.hub.yml stop -t 30
```

Caution. Do not resort to `docker kill` or `kill -9` on gunicorn workers unless the graceful path has been given at least sixty seconds to complete. An abrupt stop during a SQLite WAL flush can produce a checkpoint that requires recovery on next boot — rare, but avoidable.

Sequencing for multi-host deployments

When the deployment is split across edge and backend hosts, the sequencing of startup and shutdown matters. During startup, bring up the backend first and wait for its health endpoint to respond; only then bring up the edge. During shutdown, reverse: stop the edge first (which prevents new requests arriving), then drain the backend before stopping it.

Startup banner and log signature

On clean startup, the MedPharm process writes a short banner to stdout. The operator should recognise this banner on sight. If it is absent, the process did not start cleanly even if the container appears Up.

```
MedPharm ERP API Server v1.7.5
Binding 0.0.0.0:8080  (TLS: require)
Database: /data/medpharm_erp.db (connected, WAL mode)
Seeded: 23 models, 75 medications, 30 symptoms, 25 conditions
Worker PID 17 started.
Worker PID 18 started.
Ready to accept connections.
```

CHAPTER 10

User and Credential Administration

The MedPharm platform distinguishes two user populations: **staff** — clinicians and administrators who act on behalf of the practice — and **patients**, who access their own records through the portal or a mobile client. The two populations have distinct authentication flows, distinct credential stores, and distinct administrative procedures. This chapter covers both.

Staff accounts

Staff accounts are created from the PyQt desktop by a user with the Admin role, or via the corresponding API endpoints. Each staff account carries one of four roles: Doctor, Psychiatrist, Pharmacist, or Admin. The roles differ in the actions they permit; Admin is the superset. A minimum-necessary principle applies: grant the narrowest role that lets the staff member do their job, and nothing more.

Role	May Prescribe	May Schedule	May Dispense	May Process Claims	May Admin Users
Doctor	Yes	Yes	No	No	No
Psychiatrist	Yes	Yes	No	No	No
Pharmacist	No	No	Yes	Yes	No
Admin	Yes	Yes	Yes	Yes	Yes

Danger. The Admin role has no meaningful restrictions; it can create and destroy any record, including audit-log entries if the underlying database is accessed directly. For this reason, Admin accounts must be few and must be individually attributed. Shared Admin accounts are forbidden — they destroy the evidentiary value of the audit log.

Staff onboarding

1. In the desktop, open the User Management panel (Admin only).
2. Create the new account with a strong placeholder password generated server-side (the user will change it on first login).
3. Assign the narrowest appropriate role.
4. Communicate credentials to the user over a separate channel from the username (e.g. username by email, password by phone).
5. On the user's first login, confirm that the password-change prompt appeared and that they chose a new password.
6. Record the new account in the Commissioning Record's Staff Registry.

Staff offboarding

Staff offboarding is the moment at which operational discipline most often fails. A departed staff member whose account remains active represents a compromise vector, and compromises from stale accounts are among the most common breach vectors in health IT. The offboarding sequence is therefore short and strict: disable the account on the staff member's last working day, revoke any outstanding refresh tokens, and rotate any secret to which the staff member had access.

Patient accounts

Patients self-register through the portal by supplying four identity factors: first name, last name, date of birth, and last four digits of their Social Security Number. If those four factors match an existing patient record, the portal allows the patient to create a username and password. Patients who do not yet have a record in the system cannot register — this is by design, because the portal is not an enrolment system. New patients must first be registered by clinical staff.

Patients with forgotten passwords contact the front desk, who verify identity out-of-band and then reset the portal password via the desktop's User Management panel. The out-of-band verification is critical: a portal password reset performed on the phone without identity verification is indistinguishable, from MedPharm's point of view, from an account takeover.

JWT and refresh tokens

Mobile and desktop clients authenticate against the API using a pair of tokens: a short-lived **access token** valid for 24 hours by default, and a longer-lived **refresh token** valid for 7 days. The client includes the access token on every request; when the access token expires, the client presents the refresh token to obtain a new pair. Operators may shorten these lifetimes by adjusting the `MEDPHARM_TOKEN_EXPIRY` and `MEDPHARM_REFRESH_EXPIRY` environment variables, though shortening them also increases authentication traffic.

Revoking a refresh token — for instance, when a patient reports a lost phone — is performed by incrementing the user's token-generation counter, which invalidates every issued token for that user in one action. The operator reaches this through the User Management panel's "Revoke all sessions" button.

CHAPTER 11

Monitoring and Observability

A system that is not monitored is a system that can only tell you about its problems through the complaints of its users — the slowest, most expensive, and least reliable feedback loop available. This chapter sketches a monitoring posture appropriate for MedPharm deployments of different sizes. Small deployments may implement only the first tier; larger ones should progress through all three.

Tier 1 — The minimum viable monitoring

- **Uptime check.** A once-per-minute HTTP GET against `/api/v1/health` from an external monitoring service (Uptime Kuma, Pingdom, a cloud-native equivalent). Paging threshold: two consecutive failures.
- **Certificate expiry.** A daily check comparing the deployed certificate's notAfter date against the current date. Paging threshold: 14 days remaining.
- **Disk space.** A daily check on `/data`. Paging threshold: 10% free remaining.
- **Backup freshness.** A daily check that the latest backup file's mtime is within 26 hours. Paging threshold: miss.

Tier 2 — Structured logs and dashboards

At tier 2 the operator installs a log aggregator — Loki, Elasticsearch, CloudWatch Logs, or an equivalent — and arranges for MedPharm's stdout to flow into it. The log stream is structured JSON when `MEDPHARM_LOG_FORMAT=json`, which makes it tractable for queries. A small dashboard then visualises three metrics that catch most drift: the 95th percentile latency of `/api/v1/patient/dashboard`, the rate of HTTP 4xx responses, and the rate of HTTP 5xx responses.

Tier 3 — Application metrics

At tier 3, MedPharm emits Prometheus metrics on `/metrics` and the operator runs a Prometheus plus Grafana stack that consumes them. At this tier the operator acquires meaningful forward-looking telemetry — for example, a steadily rising P99 latency over a week may indicate a slow SQLite fragmentation problem in time to VACUUM before users notice. Tier 3 is appropriate once the deployment approaches the comfortable ceiling described in Chapter 5.

Alerting hygiene

Every alert configured in MedPharm must meet three criteria before it is allowed to page a human. It must be *actionable* — i.e., there must exist an action that the human can take in response. It must be *specific* — i.e., distinct alerts for distinct failures, not a single omnibus "something is broken" alarm. And it must be

durable — i.e., not firing on transient blips that resolve before the human reads the message. Alerts that fail any of these three criteria should be downgraded to dashboards or deleted.

Audit-log review

The AuditLog table captures every meaningful action performed by a staff user or administrative endpoint. Under HIPAA § 164.308(a)(1)(ii)(D), the operator is responsible for regular review of these records. The practice's Security Officer should review the previous week's audit log once per week, looking for: admin-role actions taken by accounts that should not have Admin; access to patient records that the actor does not have a clinical relationship with; actions occurring outside business hours; and spikes in action frequency that cannot be explained by workload.

A useful query to seed the weekly review is reproduced below. It returns all admin-role actions in the preceding seven days, grouped by actor:

```
SELECT
  actor_username,
  action,
  COUNT(*) AS n,
  MIN(timestamp) AS first_seen,
  MAX(timestamp) AS last_seen
FROM audit_log
WHERE timestamp >= datetime('now', '-7 days')
  AND actor_role = 'ADMIN'
GROUP BY actor_username, action
ORDER BY n DESC;
```

CHAPTER 12

Log Management and Rotation

Logs are the running record of a deployment's life. Well-curated logs make incident diagnosis tractable and compliance audits routine; ill-curated logs fill disks, obscure signal, and in the worst case retain protected health information past its permitted lifespan. The operator's task is to keep the logs useful without letting them become a liability.

Log sources

Source	Default Location	Typical Volume	Retention
Gunicorn (API)	/var/log/medpharm/gunicorn-api.log	≈ 20 MB / day / 1k req per day	90 days online, 2 years archive
Gunicorn (Web Portal)	/var/log/medpharm/gunicorn-web.log	≈ 5 MB / day / 100 sessions	90 days online, 2 years archive
Nginx access	/var/log/medpharm/nginx-access.log	≈ 50 MB / day / 1k req per day	30 days online, 1 year archive
Nginx error	/var/log/medpharm/nginx-error.log	≤ 1 MB / day (healthy)	1 year online
Supervisor	/var/log/medpharm/supervisord.log	minimal	1 year online
Audit log (database)	audit_log table in SQLite	variable	6 years online (HIPAA)

Rotation

On a Docker-based deployment, log rotation is performed by a `logrotate` configuration mounted into the container. The defaults rotate files larger than 50 MB, keep fourteen rotations, and compress rotations older than one day. Operators may adjust these defaults to match retention policy.

```
/var/log/medpharm/*.log {
    size 50M
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
```

Shipping logs off-host

For deployments that aggregate logs centrally, the simplest tool is the combination of `promtail` or `fluent-bit` as a sidecar, shipping each log file into the operator's preferred aggregator. An important subtlety: MedPharm's application logs never contain the content of a patient's record, but they do contain the patient ID and the action performed. Under HIPAA, the patient ID is arguably an identifier and so the log stream is PHI-adjacent. Treat the log aggregator as a subsystem that handles PHI: it requires a Business Associate Agreement with any cloud provider, and its retention policies must match the rest of the deployment.

Log review habits

Structured log review does not consist of reading every line. It consists of running a handful of saved queries and paying attention to their counts. The queries we recommend saving are: (1) error rate by endpoint, last 24 hours; (2) 95th percentile latency by endpoint, last 24 hours; (3) count of LOGIN_FAILED events grouped by source IP; (4) distinct actors performing admin actions in the last 7 days; (5) any log line containing the strings "Traceback" or "CRITICAL". The first four build situational awareness; the fifth turns up the signals that should not be present.

> **Caution.** Avoid the temptation to log protected health information in clear. MedPharm is disciplined about not writing patient names or contact details to its log files; any operator-authored extension that weakens this discipline must be removed. The presence of PHI in log streams expands the surface of the deployment in ways that retention alone cannot mitigate.

PART IV

MedPharm ERP Service Manual

Security Operations

Cryptographic posture, regulatory safeguards, and the response when something goes wrong.

CHAPTER 13

The Security Operations Handbook

Every element of MedPharm's security posture begins with an operator decision. The cryptographic defaults are strong; the authentication flows are conservative; the role model is narrow. But none of that matters if the default secrets have not been rotated, if the TLS certificate is expired, or if the Admin account's password is still *admin123*. This handbook collects the operator-facing decisions into a single place. The specific procedures live in subsequent chapters of this Part.

The hardening checklist

Every MedPharm deployment must pass this checklist before it carries real workload. A deployment that fails any row has not been hardened, regardless of how long it has been in production.

#	Control	Default	Hardened
1	MEDPHARM_JWT_SECRET	Dev default (known)	Random 256-bit value
2	MEDPHARM_SECRET_KEY	Auto-generated on first run	Set explicitly and stored in vault
3	TLS mode	auto (self-signed fallback)	require (CA-issued)
4	CORS origins	'*'	Exact hostnames only
5	Default staff accounts	All enabled	All disabled
6	Default patient accounts	5 demo patients	All deleted
7	Debug mode	False (default)	Explicitly False
8	Firewall	Open on install	Only 443 externally reachable
9	Audit log retention	Unbounded	Six years, then archived
10	Backup encryption	None by default	AES-256-GCM, keys off-host

Principle of least privilege

Least privilege governs every identity in the system. At the operating-system layer, the container runs under an unprivileged user. At the database layer, SQLite has no network surface and therefore no remote accounts — the only privilege boundary is filesystem permissions on `medpharm_erp.db`. At the application layer, the Admin role is the only role capable of creating other Admin accounts, and the audit log records every role change. The operator's task is to preserve this posture against well-intentioned drift; role inflation is remarkably common in small deployments as someone, at some point, finds it easier to hand out Admin than to refactor permissions.

Defence in depth

The deployment as a whole is resilient to any single layer failing, which is the essence of defence in depth. Nginx filters malformed HTTP; Flask's CORS middleware rejects unsolicited cross-origin requests; the JWT middleware rejects expired or mis-signed tokens; SQLAlchemy's parameterised queries neutralise SQL injection; the audit log records what did occur so that what is claimed to have occurred can be corroborated. Operators should regard any weakening of these layers — for instance, permitting a credulous CORS policy during a debugging session — as a high-risk change requiring reversal before the debugging session ends.

> **Danger.** The single most common MedPharm compromise we anticipate is the accidental retention of a relaxed CORS origin list after a development session. If you have ever run the API with `MEDPHARM_CORS_ORIGINS='*'` to debug a client problem, audit the production configuration now.

CHAPTER 14

TLS and Certificate Lifecycle

The MedPharm deployment terminates TLS on every install path. This is not optional — the HIPAA Security Rule's transmission-security standard (§ 164.312(e)(1)) mandates it for any network over which PHI may travel, which is every network MedPharm uses. This chapter covers the three phases of the certificate lifecycle — provisioning, monitoring, and rotation — and the two regimes under which each phase operates: the auto-generated self-signed regime appropriate for development, and the CA-issued regime required for production.

Provisioning a new certificate

The simplest production path is Let's Encrypt, which issues free CA-signed certificates valid for 90 days and can be automated end-to-end. The MedPharm Docker full-stack image does not ship with Certbot; the operator runs Certbot externally and drops the resulting files into the certificate volume.

```
# Obtain cert (outside the MedPharm container)
sudo certbot certonly --standalone \
  -d erp.example-clinic.com \
  --non-interactive --agree-tos --email ops@example-clinic.com

# Copy into the MedPharm volume
sudo docker volume inspect medpharm-tls
sudo cp /etc/letsencrypt/live/erp.example-clinic.com/fullchain.pem \
        /var/lib/docker/volumes/medpharm-tls/_data/fullchain.pem
sudo cp /etc/letsencrypt/live/erp.example-clinic.com/privkey.pem \
        /var/lib/docker/volumes/medpharm-tls/_data/privkey.pem

# Tighten permissions on the private key
sudo chmod 600 /var/lib/docker/volumes/medpharm-tls/_data/privkey.pem

# Reload Nginx inside the running container
docker exec medpharm-server supervisorctl signal HUP nginx
```

For private-CA deployments, the procedure is identical except that the `fullchain.pem` is produced by the private CA rather than Let's Encrypt. Be sure to include the intermediate chain; mobile clients are particularly intolerant of truncated chains.

Monitoring validity

A certificate that expires unnoticed is a service outage. MedPharm does not monitor its own certificate — the responsibility lies with the operator. A single command, run daily from cron, suffices:

```
#!/bin/bash
# /usr/local/bin/medpharm-cert-check
```

```
DAYS=$(echo | openssl s_client -servername erp.example-clinic.com \
  -connect erp.example-clinic.com:443 2>/dev/null | \
  openssl x509 -noout -enddate | cut -d= -f2)
EPOCH=$(date -d "$DAYS" +%s)
NOW=$(date +%s)
REMAINING=$(( (EPOCH - NOW) / 86400 ))
if [ "$REMAINING" -lt 14 ]; then
  echo "MedPharm TLS certificate expires in $REMAINING days" | \
    mail -s "MedPharm TLS expiry warning" ops@example-clinic.com
fi
```

Rotation

Rotating a certificate consists of replacing the two files in the TLS volume and signalling Nginx to reload its configuration. It must be possible to do this without interrupting the service. MedPharm's configuration supports this: Nginx's `SIGHUP` handler reloads configuration without dropping active connections. The rotation procedure is therefore:

1. Stage the new `fullchain.pem` and `privkey.pem` in a scratch directory.
2. Verify the new certificate matches the deployed hostname (`openssl x509 -in fullchain.pem -noout -text | grep DNS:`).
3. Replace the files in the TLS volume atomically (write to a temporary name, then `mv`).
4. Signal Nginx: `docker exec medpharm-server supervisorctl signal HUP nginx`.
5. Verify from an external client that the new certificate is being served and that its fingerprint matches the expected value.

Note. If Let's Encrypt is used with the renewal-hook mechanism, rotation becomes fully automatic. Add a short script to the certbot `--deploy-hook` that copies the new files into the MedPharm volume and signals Nginx. Certbot then performs rotations without operator involvement, and the operator's task reduces to periodic verification.

CHAPTER 15

Secret Rotation and Key Hygiene

Every cryptographic system degrades if its secrets do not move. A JWT signing key that has been in use for five years represents a slow-building risk: with each passing year the key sits on one more departed engineer's laptop backup, in one more forgotten ticket system, on one more decommissioned host. Key rotation converts that risk, cheaply and predictably, into maintenance. This chapter describes MedPharm's two principal secrets and the rotation cadences that are appropriate for each.

The JWT signing key

MedPharm signs access and refresh tokens using HMAC-SHA256 with the `MEDPHARM_JWT_SECRET` environment variable as the key. Rotating this key invalidates every outstanding token system-wide; every currently-logged-in client will be forced to log in again. This is aggressive but predictable, and the correct response to any suspicion that the key has leaked.

1. Generate a new key: `openssl rand -base64 48`.
2. Stage the new value in the environment file used by the deployment.
3. Restart the container: `docker compose restart api`.
4. Verify a fresh login succeeds.
5. Verify previously-held tokens are rejected (test from a second browser).
6. Record the rotation event in the Security Officer's log with the date and the fingerprint (not the value) of the new key.

The Flask session key

The `MEDPHARM_SECRET_KEY` secures Flask session cookies used by the patient portal. Rotating it invalidates every existing portal session; users will be redirected to the login page on their next request. This is a milder event than JWT rotation (portal users are generally fewer and more tolerant) and can be done quarterly without operational burden.

Rotation cadences

Secret	Cadence (Routine)	Cadence (After Incident)
MEDPHARM_JWT_SECRET	Every 12 months	Immediately on suspicion of leak
MEDPHARM_SECRET_KEY	Every 3 months	Immediately on suspicion of leak
TLS private key	On every cert renewal	Immediately on suspicion of leak
Admin account passwords	Every 6 months	Immediately on offboarding

Secret	Cadence (Routine)	Cadence (After Incident)
Backup encryption key	Every 12 months	Immediately on storage-provider change
Database filesystem permissions	Validate weekly	After any install/upgrade

Storing secrets

MedPharm's environment variables are a low bar — they work, but they are not a vault. For anything beyond the smallest deployment, the operator should keep the authoritative copy of each secret in a proper vault (HashiCorp Vault, AWS Secrets Manager, 1Password Business), and arrange for the deployment's orchestrator to fetch the current value at startup. This keeps the secrets out of container images, out of Git history, and out of shell history, which are the three places they most commonly leak from.

Danger. Never commit a secret to Git, even to a private repository, even briefly, even to be rebased away later. Once a secret has entered source control, it must be treated as leaked. Rotate the secret first, rewrite the history second; in that order, not the reverse.

CHAPTER 16

HIPAA Operational Controls

HIPAA compliance is a property of an *organisation*, not of a *software package* — a common misunderstanding when procuring clinical software. MedPharm ERP is designed to make organisational compliance tractable: its technical posture supplies most of the controls that the Security Rule's technical safeguards demand. The operator is still responsible for the administrative and physical safeguards, and for using MedPharm in a manner consistent with its designed posture.

Mapping MedPharm controls to the Security Rule

The table below enumerates the controls in the Security Rule's technical safeguards (§ 164.312) and the MedPharm feature or procedure that implements each. This mapping is not a substitute for a full compliance audit but it provides the skeleton around which one is conducted.

§ Reference	Control Requirement	MedPharm Implementation
164.312(a)(1)	Access control	Role-based staff accounts; JWT-authenticated API; four-factor patient identity verification
164.312(a)(2)(i)	Unique user identification	Each staff and patient has a distinct username; shared accounts forbidden
164.312(a)(2)(iii)	Automatic logoff	JWT expires after 24 hours; desktop application session timeout configurable
164.312(a)(2)(iv)	Encryption and decryption	Patient passwords hashed with PBKDF2-SHA256; Windows client uses DPAPI; iOS/macOS use Keychain
164.312(b)	Audit controls	AuditLog table captures all sensitive actions; weekly Security Officer review mandated by this manual
164.312(c)	Integrity	SQLite journaling; backup verification; TLS prevents in-transit tampering
164.312(d)	Person or entity authentication	Four-factor registration; password + JWT for subsequent access
164.312(e)(1)	Transmission security	TLS enabled by default on every install path; self-signed fallback forbidden in production

Administrative safeguards

The Security Rule's administrative safeguards (§ 164.308) require written policies that the operator is responsible for implementing, not just reading. The operator should have, on file, at least the following named documents: a **Sanction Policy** for staff who violate the practice's security procedures; an **Information System Activity Review** procedure, referencing this manual's audit-log chapter; a **Workforce Clearance** procedure for new hires; a **Termination Procedure** mirroring the offboarding steps

in Chapter 10; a **Data Backup Plan** mirroring Chapter 19; a **Disaster Recovery Plan** mirroring Chapter 32; and an **Emergency Mode Operation Plan** describing how clinical work continues during a prolonged outage.

Business Associate Agreements

Every third party who handles PHI on the practice's behalf must be under a signed Business Associate Agreement. For MedPharm deployments the typical list is: the cloud hosting provider; the backup storage provider; the TLS certificate automation provider (if any); the log aggregation provider (if any); and any contracted operator. Enlightec Ltd. supplies a template BAA at `docs/BAA_TEMPLATE.md` suitable for use with third-party operators; it should be reviewed by counsel before execution.

Legend. A BAA that has been signed but not filed is not, in a regulatory sense, in force. Build a filing discipline around BAAs early — their value lies in being producible on demand during an audit.

CHAPTER 17

Incident Response Playbook

An incident is any event that, confirmed, would constitute a compromise of confidentiality, integrity, or availability. Suspicion is treated as an incident; triage downgrades suspicions that do not pan out. This disposition — lean toward treating — is deliberate. Under-reacting is expensive in ways that over-reacting is not.

Classification

Severity	Examples	Response Time
SEV-1 (Critical)	Confirmed PHI disclosure; ransomware detected; database compromised	Immediate; page on-call at any hour
SEV-2 (High)	Active unauthorised access attempt; auth service outage; certificate expired	Within 1 hour during business hours, within 4 hours otherwise
SEV-3 (Medium)	Degraded performance; intermittent 5xx errors; backup job failure	Same business day
SEV-4 (Low)	Non-user-facing log noise; minor misconfiguration	Within one week

The first-hour playbook

When a suspected SEV-1 is declared, the first hour determines much of the outcome. Panic is expensive and so is delay. The following sequence trades a small amount of time for a large amount of clarity.

1. **Declare.** Use the phrase "I am declaring a SEV-1 incident" in the team chat channel. The words matter; they activate the rest of the playbook and anchor a timestamp.
2. **Page.** Page the Security Officer, the Privacy Officer, and the deputy System Operator. Assume the first person reached will be the incident commander.
3. **Preserve.** Do not restart the affected system yet. Snapshot the filesystem of the affected host; export the audit log as it stands; capture the output of `docker ps`, `ss -tlnp`, and `last`.
4. **Contain.** If the incident involves unauthorised access, rotate MEDPHARM_JWT_SECRET and MEDPHARM_SECRET_KEY to force all sessions to re-authenticate. If it involves confirmed data exfiltration, firewall off external traffic to the affected host.
5. **Communicate.** The incident commander publishes a short status note every 30 minutes even if no new information has emerged. Silence during an incident is interpreted as progress by people downstream, who then make commitments based on a false picture.
6. **Remediate.** Only once the first five steps are complete does the remediation begin. Restoration from backup, image rebuilds, credential resets — all proceed from the preserved snapshot, not the live system.

7 **Post-mortem.** Within five working days of resolution, a blameless post-mortem document is produced, circulated, and filed alongside the Commissioning Record.

Breach notification

A confirmed breach of unsecured PHI triggers obligations under the HIPAA Breach Notification Rule (§ 164.400–§ 164.414). The operator is not the appropriate party to decide whether a breach notification is required — that decision belongs to the Privacy Officer, on counsel's advice. The operator's duty is to produce the information on which that decision rests: what was accessed, by whom, when, and whether the evidence indicates confidentiality was compromised or merely could have been. The AuditLog, preserved at step 3 of the playbook, is the primary source of this information.

A breach-notification template is supplied at `docs/BREACH_NOTIFICATION.md` in the distribution. The Privacy Officer should review it annually and ensure that contact details for the HHS Office for Civil Rights and any applicable state authorities are current.

Danger. HIPAA requires notification within sixty days of discovery of a breach affecting 500 or more individuals; state laws may require faster or broader notification. Do not assume the federal timeline is the binding one without checking against the jurisdictions in which the practice operates.

PART V

MedPharm ERP Service Manual

Data Stewardship

Looking after the data — in life, across backups, and into its eventual disposal.

CHAPTER 18

Database Administration

SQLite's charm as a backend for MedPharm is that it requires so little administration. There is no server daemon to monitor, no authentication to manage, no network surface to lock down, and no cluster to keep in consensus. The trade for that simplicity is that SQLite's performance envelope is narrower than a full-blown RDBMS, and certain operations that a DBA would take for granted elsewhere (online schema changes, concurrent writers) have to be approached carefully here. This chapter is the operator's working familiarity with that envelope.

Connection mode and journaling

MedPharm opens its SQLite database in Write-Ahead Logging (WAL) mode. Under WAL, writers do not block readers and readers do not block writers — which is why a medium practice's fifteen concurrent desktop staff can coexist with fifty mobile clients on a single SQLite file without queueing. WAL does, however, require that the `-wal` and `-shm` sidecar files sit alongside the main database file on the same filesystem. Operators who have ever copied a running database file without its sidecars are familiar with the confused results; the backup procedures in Chapter 19 handle this correctly.

Inspecting the database from the shell

The SQLite command-line tool is the operator's most useful instrument for direct inspection. Install it on the host or run it from inside the container.

```
# From inside the container
docker exec -it medpharm-server sqlite3 /data/medpharm_erp.db

sqlite> .tables           -- list all tables
sqlite> .schema patients  -- show CREATE TABLE for patients
sqlite> .mode column
sqlite> .headers on
sqlite> SELECT COUNT(*) AS patients FROM patients;
sqlite> SELECT COUNT(*) AS rx FROM prescriptions WHERE created_at > date('now', '-7
days');
sqlite> PRAGMA integrity_check;
sqlite> PRAGMA journal_mode;   -- should report 'wal'
sqlite> PRAGMA wal_checkpoint(TRUNCATE);
sqlite> .quit
```

Caution. The operator must *never* open the MedPharm database in a GUI tool that makes auto-modifications (for example, one that silently upgrades schema or creates metadata tables). Use read-only tools (the built-in sqlite3 with `--readonly`, or DB Browser for SQLite's "Open Read-Only" option) unless an edit is specifically required.

Routine PRAGMA maintenance

A small number of PRAGMA-driven maintenance operations keep a SQLite database healthy as it grows. MedPharm's workload produces very low write amplification, so these need not be run frequently, but they should not be neglected indefinitely.

PRAGMA / Command	Purpose	Cadence
integrity_check	Verify database has no corruption	Monthly
wal_checkpoint(TRUNCATE)	Merge WAL into main DB, free WAL file	Weekly during low traffic
ANALYZE	Refresh query planner statistics	After bulk inserts
VACUUM	Reclaim deleted space; defragments file	Quarterly; requires exclusive access
PRAGMA journal_size_limit	Cap WAL growth	Set once at commission
PRAGMA page_size	Tune for workload	Set once at database creation

Schema migrations

MedPharm upgrades occasionally carry schema changes. The project does not ship a formal migration framework (Alembic or similar); instead, each release that introduces a schema change bundles an idempotent upgrade script that the installer runs automatically. Operators should, before every upgrade, take a backup and inspect the release notes to understand whether the upgrade alters schema. An upgrade that adds a column to a million-row table may take several minutes, during which the application is stopped. This window is declared in the release notes for every schema-bearing release.

Useful queries for operators

The following queries are used routinely by the operator to answer common questions about the state of the system. They are safe — they perform only reads — and should be saved in a file that the operator can source on demand.

```
-- How many active patients?
SELECT COUNT(*) FROM patients WHERE status = 'ACTIVE';

-- Prescriptions written today, by provider
SELECT p.full_name, COUNT(*) AS rx_today
FROM prescriptions r JOIN providers p ON r.provider_id = p.id
WHERE date(r.created_at) = date('now')
GROUP BY p.full_name ORDER BY rx_today DESC;

-- Outstanding balance, by age bucket
SELECT
  CASE
    WHEN julianday('now') - julianday(due_date) <  30 THEN '0-30 days'
    WHEN julianday('now') - julianday(due_date) <  60 THEN '30-60 days'
```

```
    WHEN julianday('now') - julianday(due_date) <  90 THEN '60-90 days'
    ELSE '90+ days'
  END AS age_bucket,
  SUM(balance) AS total
FROM invoices WHERE status != 'PAID'
GROUP BY age_bucket;

-- Claims stuck in IN_REVIEW longer than 10 days
SELECT id, invoice_id, submitted_at, julianday('now') - julianday(submitted_at) AS days
FROM insurance_claims
WHERE status = 'IN_REVIEW' AND days > 10
ORDER BY days DESC;
```

CHAPTER 19

Backup Strategy and Procedures

The backup strategy described here is designed around three guarantees: that a backup exists which is at most 24 hours old; that the backup is restorable without operator surprise; and that the backup is stored in a location whose compromise is uncorrelated with the compromise of the primary. Together these guarantees convert most plausible failure modes into inconvenience rather than disaster.

The 3-2-1 rule, restated

The common "3-2-1" backup rule — three copies of the data, on two different media, with one copy off-site — applies directly to MedPharm. The three copies are: (1) the live database, (2) the on-host nightly snapshot, (3) the off-host replicated copy. The two media are the application's persistent volume and the backup target. The off-site requirement is met by the off-host copy, which must not sit in the same failure domain as the primary (not on the same VM's disk, not in the same cloud region, not on the same physical site).

The nightly snapshot

The primary backup mechanism is a nightly cron job that uses SQLite's online backup API (via `sqlite3 .backup`) to produce a consistent snapshot without stopping the application. The command is safe to run while the database is being written to: it respects the WAL and produces a file that is internally consistent as of the moment it began.

```
#!/bin/bash
# /usr/local/bin/medpharm-backup.sh
set -eu
TIMESTAMP=$(date -u +%Y%m%dT%H%M%SZ)
BACKUP_DIR=/var/backups/medpharm
BACKUP_FILE="${BACKUP_DIR}/medpharm_${TIMESTAMP}.db"

mkdir -p "$BACKUP_DIR"

# Consistent hot backup
docker exec medpharm-server \
  sqlite3 /data/medpharm_erp.db ".backup '$BACKUP_FILE'"

# Compress and encrypt
gzip -9 "$BACKUP_FILE"
gpg --batch --yes --recipient ops@example-clinic.com \
    --encrypt "${BACKUP_FILE}.gz"
shred -u "${BACKUP_FILE}.gz"

# Ship to off-site target
aws s3 cp "${BACKUP_FILE}.gz.gpg" s3://medpharm-backups/nightly/
```

```
# Retain 30 days on-host
find "$BACKUP_DIR" -name 'medpharm_*.gz.gpg' -mtime +30 -delete
```

Danger. The backup script above encrypts with GPG using a public key whose private counterpart is held off-host. If you lose access to that private key, every backup becomes unrecoverable. Maintain at least two independent custodians of the private key, and test decryption from both custodians quarterly.

Schedule

Cron is sufficient for most deployments. The nightly job runs at 02:15 local time — chosen to be after any log-rotation activity and before the first inbound traffic of the business day, with a comfortable margin for completion even at the largest practice sizes.

```
# /etc/cron.d/medpharm
15 2 * * * root /usr/local/bin/medpharm-backup.sh \
    >> /var/log/medpharm/backup.log 2>&1
# Send a heartbeat to external monitoring so we know it ran
20 2 * * * root curl -fsS --retry 3 \
    https://hc-ping.com/<uuid> > /dev/null
```

Verification

A backup that has never been restored is not yet a backup — it is a promissory note. Chapter 20 describes the restore procedure in detail; here we note only that the Wednesday slot of the weekly rotation (Chapter 8) is reserved for the operator to execute a full restore onto a throw-away host. The result of each restore is recorded in the operator's journal. A restore that completes cleanly once is not proof that the next one will; regular rehearsal is what promotes promissory notes to backups.

Retention

Tier	Location	Retention	Encryption
Hot (last 7 days)	On-host /var/backups	7 days	GPG public-key
Warm (last 90 days)	Off-host object storage	90 days	GPG public-key + provider-side at rest
Cold (archive)	Glacier / Archive tier	6 years (HIPAA minimum)	GPG public-key

CHAPTER 20

Restore and Point-in-Time Recovery

Restore is the other half of backup and the half that operators rehearse less often than they should. This chapter covers two scenarios: the routine full restore onto a fresh host (Wednesday's weekly task), and the point-in-time restore performed during an incident. Both are exercises that, given the stakes, reward patience over speed. Read the whole procedure before beginning.

Routine full restore (rehearsal)

1. Provision a fresh VM of the same profile as the primary.
2. Install MedPharm using `./install.sh --docker-server` (do not start the container yet — use `up --no-start` if necessary).
3. Fetch the latest backup from the off-site target onto the fresh host.
4. Decrypt: `gpg --decrypt backup.db.gz.gpg > backup.db.gz`.
5. Decompress: `gunzip backup.db.gz`.
6. Replace the placeholder database on the new host's data volume with the decrypted file.
7. Start the container, and watch startup logs for the integrity banner.
8. Log in via the desktop with a known staff account and confirm expected data is present.
9. Destroy the rehearsal host. Record the rehearsal outcome in the operator's journal.

Incident restore (production)

When a production restore is the correct response to an incident — typically after confirmed data corruption or ransomware — the procedure is essentially the same as the rehearsal, with three additional considerations.

- **Preserve the compromised host.** Do not reimage or reformat the compromised host; forensic analysis of it may be required later.
- **Select the correct recovery point.** Choose the most recent backup that predates the corruption. If the timing is unclear, restore the most recent backup to a sandbox, inspect it, and only then decide whether to use it.
- **Communicate the recovery-point objective.** The restore is the moment at which the operator must tell clinical staff what will be lost — typically, prescriptions written between the last backup and the incident. Those items must be manually re-entered; the practice must know in advance that this will be necessary.

Recovery-point and recovery-time objectives

Under the default backup regime — nightly at 02:15 — the Recovery-Point Objective (RPO) is 24 hours: the most the practice can lose is one business day of work performed after the last backup. The Recovery-Time Objective (RTO) is roughly one hour, dominated by the time to provision a fresh VM, fetch the backup, decrypt, and restart the service. Organisations with tighter RPO or RTO requirements should refer to Chapter 33 for the architectures that support them — principally, more frequent backups and a warm stand-by host.

Note. RPO and RTO are organisational commitments, not technical facts. They should be documented in the practice's Disaster Recovery Plan and the infrastructure sized to meet them. A 24-hour RPO is adequate for most small practices; a practice whose clinical workflow cannot tolerate losing a day's prescriptions must invest in more frequent checkpoints.

CHAPTER 21

Data Retention, Archival, and Disposal

Health records do not merely age; they migrate, across their lifetime, through several legal and practical regimes. A prescription written last Tuesday is operational data, consulted often. A prescription written seven years ago is an archival record, consulted rarely but required by law to be producible. A prescription written thirty years ago for a patient who has died is, in most jurisdictions, a record that may — and sometimes must — be disposed of. The operator's job is to move data across these regimes in a controlled way.

Retention periods

Record Type	Minimum Retention	Basis
Adult clinical records	6 years after last encounter	HIPAA § 164.530(j); state law may extend
Pediatric clinical records	Age of majority + state minimum	State statute; commonly 21 years of age
Prescription records (DEA)	2 years	21 CFR 1304; may conflict with longer HIPAA term
Controlled-substance inventories	2 years	21 CFR 1304
Audit logs	6 years	HIPAA § 164.316(b)(2)(i)
Billing / financial records	7 years	IRS guidance; state revenue department
Business Associate Agreements	6 years after termination	HIPAA § 164.316(b)(2)(i)

The longest applicable retention period governs. A prescription record may therefore be retained for six years by HIPAA, even though the DEA minimum is two. When the legal basis for retention ends, the operator does not merely become free to delete — they must actively dispose, because continued retention of PHI past its minimum-necessary lifespan is, under HIPAA, itself a compliance failure.

Archival

MedPharm does not perform automatic archival. Operators must implement a manual or scripted archival process that moves records whose operational relevance has expired but whose retention requirement has not. A practical convention is to export inactive patients' records — those with no encounter in the past 24 months — into an encrypted archive file annually, and then either keep them in place (they are cheap) or remove them from the live database and retain only the archive. The trade is familiar: in-place retention is simpler; archival reduces the blast radius of a production compromise.

Disposal

When the retention window closes, disposal must be irreversible and documented. The disposal record names the record, the date it was disposed, the mechanism, and the operator who performed it. Under HIPAA § 164.530(c), disposal must 'render the information essentially unreadable, indecipherable, and otherwise cannot be reconstructed.' For SQLite data this means a DELETE followed by a VACUUM (which overwrites the freed pages) on the live database, and cryptographic destruction of any surviving backup — typically by destroying the GPG private key that decrypts it, if the backup is encrypted.

Caution. A naive DELETE in SQLite does not overwrite the underlying storage; the data can be recovered from the file. If you are disposing of PHI that must be destroyed rather than merely hidden, run `VACUUM` afterwards, and if the deployment is hosted on SSD, note that VACUUM does not guarantee destruction at the physical layer. For regulatory destruction, rotate the entire database onto new storage.

PART VI

MedPharm ERP Service Manual

Maintenance and Change

Keeping the deployment current, safe, and well-documented as it evolves.

CHAPTER 22

The Scheduled Maintenance Calendar

Planned maintenance is how a MedPharm deployment remains boring — which, for a system carrying patient workload, is the highest compliment available. The calendar that follows is the one we would have a greenfield deployment adopt on its first day in service. It is not prescriptive; operators will adapt it to their local constraints. But it is complete: every task necessary to keep the deployment healthy appears somewhere in it.

Daily

Daily work is entirely absorbed by the walk-around described in Chapter 8. It is brief, it is uninterruptible, and it is best done at the same time each day so that deviations become visible as calendar anomalies rather than as surprises.

Weekly

Each day of the working week carries a single longer task, listed again for convenience. The Sunday slot is empty by design; an operator who never unplugs is an operator whose judgement will fray.

Day	Task
Monday	Full-week audit log review
Tuesday	Log file rotation / archival
Wednesday	Restore rehearsal onto throw-away host
Thursday	Upstream advisory review
Friday	Non-urgent security patches applied
Saturday	TLS cert expiry check
Sunday	Rest; on-call only

Monthly

- Run `PRAGMA integrity_check` on the database and file the result.
- Reconcile the Staff Registry against the currently active staff accounts.
- Review the InsuranceProvider seed data for additions or discontinuations.
- Review the medication formulary against current prescribing practice.
- Conduct the "tabletop" half of a disaster-recovery walk-through (Chapter 32) — a fifteen-minute chat, not an actual failover.

Quarterly

- Run `VACUUM` on the database during a scheduled maintenance window. Confirm database size shrinks.
- Rotate MEDPHARM_SECRET_KEY (Chapter 15).
- Verify backup encryption keys with all custodians.
- Review and update the escalation matrix.
- Conduct a full-restore rehearsal from the oldest retained backup, not the newest.

Annual

- Rotate MEDPHARM_JWT_SECRET.
- Rotate the backup encryption key; re-encrypt the most recent backups with the new key.
- Conduct a full live disaster-recovery exercise on a scheduled maintenance weekend.
- Review all Business Associate Agreements for expiry and continued relevance.
- Reassess sizing envelope against the previous twelve months' growth.
- Commission an external penetration test of the deployment.

Note. A calendar of maintenance work only helps if it is kept. The simplest trick is to put every item on the real calendar — operators' shared work calendar, not a separate "ops calendar" that nobody reads. The best disciplines are the ones that feel, after six months, like habits rather than plans.

CHAPTER 23

Patch and Upgrade Procedures

MedPharm releases fall into two classes: **point releases**, which are conservative bugfixes, dependency updates, and strictly backward-compatible feature additions; and **minor releases**, which may carry schema changes and subtle behavioural differences. Both classes are distinguished by the third and second components of the semantic version, respectively. Major releases — which would carry breaking API changes — have not yet occurred and will be announced well in advance of release.

The upgrade discipline

Four habits keep upgrades uneventful. First, always read the release notes before beginning; they are short and they are, typically, the only place that schema changes and operator-facing behaviour shifts are declared. Second, always take a backup immediately before the upgrade. Third, upgrade staging before production, and let at least a day elapse between them. Fourth, have a rollback plan ready before you begin; an upgrade with no rollback is a one-way door.

The standard upgrade procedure (Docker)

1. Read release notes at `docs/CHANGELOG.md` or on the upstream release page.
2. Announce the maintenance window to clinical staff (even if it will be brief, even for a point release).
3. Take a fresh backup: `medpharm-backup.sh`.
4. Verify the backup is readable.
5. Pull the new image: `docker compose pull`.
6. Stop the current container: `docker compose stop`.
7. Start the new container: `docker compose up -d`.
8. Observe the startup banner and watch the logs for five minutes.
9. Run the validation matrix from Chapter 7.
10. Monitor closely for 24 hours; file any new log signatures that appear.

The rollback plan

The rollback plan is simple and must be rehearsed: stop the new container; retag the previous image as `enlightec/medpharm-server:previous`; restart against that image; restore the database from the pre-upgrade backup if the upgrade altered schema. For a point release that did not touch schema, the rollback is a minute's work. For a minor release that did, it is the better part of an hour. Time both rollbacks during the staging rehearsal so that the production rollback's cost is known in advance.

Danger. A minor release that altered schema cannot be rolled back in place by simply restoring the old image; the database's post-migration state will no longer match the old application's expectations. The rollback path for schema-altering releases *requires* restoring the pre-upgrade backup. This is why the step sequence in this chapter insists on a backup immediately before every upgrade, without exception.

Staging-to-production cadence

Staging is a first-class environment: it has its own certificate, its own database seeded with de-identified (or synthetic) data, and its own backups. Operators should refuse the temptation to test upgrades against production on the grounds that staging "isn't really the same." Where staging is not really the same, the correct response is to make it closer, not to bypass it.

CHAPTER 24

Dependency Hygiene

MedPharm depends on a small but not trivial set of external software: Python, its standard library, Flask, SQLAlchemy, gunicorn, and perhaps a dozen smaller packages on the server side; Kotlin, Swift, C#, and their platform SDKs on the client side. Each dependency is a channel through which security advisories flow into the operator's working week. This chapter describes how to keep that channel orderly.

Advisory sources to monitor

Ecosystem	Source	Frequency
Python	Python Security Response Team (PSRT); python-security mailing list	As published
Flask / SQLAlchemy	PyPA Advisory DB; GitHub Security Advisories on the respective repos	As published
Ubuntu base image	Ubuntu Security Notices (USN)	Daily digest
Nginx	Nginx security advisories	As published
Kotlin / Gradle	JetBrains security bulletins	As published
Swift / Xcode	Apple Developer security announcements	Per release
NuGet (.NET)	NuGet Advisory DB; Microsoft Security Updates	As published
Docker / runc	Docker and containerd security advisories	As published

Triaging advisories

Most advisories will not apply. The question to ask of each is: does the affected functionality appear in MedPharm's dependency tree, and if so, on the execution path? A CVE in Flask's `send_from_directory` helper, for example, does not apply to a MedPharm deployment that never calls that helper. Record the triage decision in the operator's journal even when the decision is "not applicable" — future you, or the auditor, will want to know why an advisory was left alone.

Applying updates

Dependency updates are applied through the usual upgrade procedure (Chapter 23). The important discipline is to *separate* dependency updates from feature changes: a release that bumps Flask from 3.0.0 to 3.0.3 should not also introduce a new endpoint. Keeping these concerns separate preserves the ability to bisect when a dependency update introduces a regression, which, rarely, it will.

End-of-life watch

At longer cadence, track the end-of-life dates of the packages on which MedPharm depends. Python minor versions reach end of life roughly five years after release; the Ubuntu LTS base image carries a ten-year support window. Treat an approaching EOL as a project: a Python 3.10 deployment in its final year should have a Python 3.12 or later migration on the schedule.

CHAPTER 25

Change Management Workflow

Every operator intervention into a production MedPharm deployment is a change, and every change either succeeds, fails, or succeeds in a way that is only apparent later. A minimal change-management workflow makes this tractable: it ensures that the operator proposes before they act, that the action is recorded, and that the effect is verified. The workflow described here is deliberately simple. Elaborate change control is a luxury good; a simple one, consistently applied, is a necessity.

The four-stage workflow

1. **Propose.** The operator writes a short change request: what will change, why, when, and how the success of the change will be evaluated. For routine changes (patches, certificate rotations) a template suffices.
2. **Approve.** A second operator (or the clinical administrator, for behaviour-affecting changes) approves. Self-approval is permitted only for the lowest-risk, reversible routine tasks; the operator's journal records these for retrospective review.
3. **Execute.** The change is applied, and the operator observes its immediate effect. Time-stamp the start and end of the window.
4. **Verify.** The operator runs the agreed-upon evaluation criteria. A change that cannot be verified is not complete; if verification is deferred, say so explicitly and schedule it.

The change register

A change register — a simple append-only log — is the most effective tool available to a small operations team. Every change, however small, goes in the register. Entries are short: date, author, approver, description, outcome. After a year the register becomes an invaluable archaeological resource: when a subtle problem emerges that coincides with an old change, the register makes the correlation visible in thirty seconds.

Date (UTC)	Author	Approver	Change	Outcome
2026-04-15T02:30Z	R. Stillwell	self	Rotate MEDPHARM_SECRET_KEY	OK
2026-04-14T14:05Z	R. Stillwell	C. Mahler	Upgrade medpharm-server 1.7.2 → 1.7.3	OK; 4 min window
2026-04-11T09:12Z	C. Mahler	self	Replace TLS cert (Let's Encrypt auto-renewal)	OK
2026-04-04T19:00Z	R. Stillwell	C. Mahler	Apply USN-6723-1 kernel update; host reboot	OK; 2 min window
2026-03-30T03:15Z	(cron)	n/a	Nightly backup (automated)	OK

Emergency changes

Emergency changes — those that must be applied immediately to preserve service or security — bypass the Propose and Approve stages by necessity but never the Execute and Verify stages. An emergency change's entry in the register notes, explicitly, that it was an emergency. Within 72 hours, the operator writes a retrospective justification and obtains retroactive approval. This preserves the register's value as a record of who knew what, when — which is exactly what you want when an auditor asks.

PART VII

MedPharm ERP Service Manual

Fault Diagnosis

Method, symptom catalogues, and the recipes that most often close tickets.

CHAPTER 26

Troubleshooting Methodology

Diagnosis under pressure is less a matter of knowing facts than a matter of holding a method. Facts are useful, but facts without method produce shotgun debugging — the tendency to change several things at once in the hope that one of them was the right one. Shotgun debugging lengthens incidents and obscures root causes. The method offered here is neither novel nor elaborate; it simply insists on being followed.

The five questions

Before touching anything, answer five questions. If you cannot answer any one of them, the next investigative action is to find the answer, not to apply a fix.

1. **What are the specific symptoms?** Not "it's broken," but "POST /api/v1/auth/login/patient returns 502 approximately 40% of the time since 14:20 UTC."
2. **What changed?** An upgrade? A new client version? A configuration edit? A dependency update? An unrelated change on the host? If nothing intentional changed, something unintentional did — advance by looking for it.
3. **Who is affected?** Everyone, or one user? All clients, or just the iOS client? From one subnet only, or universally? The blast radius tells you where to look.
4. **What is the first point in the request path at which the fault appears?** Nginx logs, API logs, database logs, client logs. Work outside-in; the fault is usually closer to the symptom than it first appears.
5. **What is the smallest reproduction?** If you can reproduce the fault with a single curl command, you can experiment quickly and without disruption. If you cannot reproduce it, you are not yet ready to fix it.

The change axis

Most faults in a production deployment are correlated with a recent change. The change register (Chapter 25) is therefore the first artefact to consult during diagnosis. "What's the last thing we touched?" is the single most productive question in operations. Pair it with "what was its rollback?" and many incidents dissolve in less than half an hour.

Eliminate, don't accumulate

Apply changes one at a time during diagnosis, and revert each one if it does not resolve the problem. Every change-on-top-of-change lengthens the back-out path. If you must apply multiple changes (rare, but possible), document the order so that the rollback exists before the first change is applied.

Caution. Diagnostic actions taken during an incident are changes, and they belong in the change register too — even the temporary ones. A debugging log level raised to TRACE at 02:00 that is still in place at 10:00 the next morning is a common source of follow-on confusion.

CHAPTER 27

Symptom Directory — Server-Side

The symptoms catalogued below are those most likely to occur on a MedPharm server during its first five years of life. Each entry states the observable symptom, lists probable causes in approximate order of frequency, points to the next diagnostic step, and names the chapter in which the full procedure lives. The list is deliberately incomplete — a comprehensive catalogue would be forgotten on sight — but it covers the cases that repeatedly recur in real deployments.

API returns 502 Bad Gateway intermittently

Probable causes, by frequency: Gunicorn workers are being killed by an OOM condition; Nginx proxy timeout is shorter than a slow query's response time; the Gunicorn socket is being hot-rotated during a restart; the database is locked by a long-running write. **First diagnostic step**: `docker exec medpharm-server supervisorctl status` to see whether the API worker has restart counts; then `docker logs --tail=200 medpharm-server` looking for SIGKILL or *Worker timeout*.

API returns 401 Unauthorized on all requests

Probable causes: JWT secret was rotated but a subset of gunicorn workers holds the old one; clients have cached expired tokens; system clock has drifted beyond the JWT's clock-skew tolerance. **First diagnostic step**: run `date -u` on the server and on an affected client; if they differ by more than 30 seconds, resynchronise NTP first and retest.

API returns 500 Internal Server Error

Probable causes: unhandled exception in a route handler, usually revealed by a recent code change; database connection error; third-party dependency regression. **First diagnostic step**: `docker logs --tail=500 medpharm-server | grep Traceback` . Python tracebacks are rare and typically conclusive.

Database is locked

Probable causes: a long-running manual query holding a write transaction; a stuck gunicorn worker that has not relinquished a write lock; two processes opening the file with incompatible journal modes. **First diagnostic step**: `docker exec medpharm-server lsof | grep medpharm_erp.db` to enumerate holders, then consider terminating the longest-running holder. Be careful: a terminated writer mid-transaction leaves SQLite in a state it must recover from on next open.

Disk fills unexpectedly

Probable causes: log rotation is not running; the WAL file has grown without checkpoint (typically because a reader is holding a long-running transaction); a backup directory is not being pruned; a Docker overlay is leaking. **First diagnostic step**: `du -sh /var/log/medpharm /var/backups/medpharm /data` to locate the growth. Most often the answer is log rotation.

Nginx returns 504 Gateway Timeout

Probable causes: a heavy analytics query is running and exceeding Nginx's `proxy_read_timeout`; the API process is CPU-saturated; a dependency call (e.g. a third-party insurance-claim processor) is hanging. **First diagnostic step**: correlate the timeout against API logs — the same request should appear in both streams.

Container restarts in a loop

Probable causes: missing required environment variable (most commonly `MEDPHARM_JWT_SECRET` left unset in a production deployment with `MEDPHARM_TLS_MODE=require`); TLS certificate files missing or invalid; database file permission mismatch after a *chown* accident. **First diagnostic step**: `docker logs medpharm-server` — the entrypoint script prints a diagnostic message before exiting.

Portal login succeeds, dashboard blank

Probable causes: a JavaScript error on the portal bundle; static assets not being served (typically an Nginx location block misconfigured after a template edit); session cookie domain does not match the host the client connects to. **First diagnostic step**: open browser developer tools and look for 404s in the Network panel or red messages in the Console.

Drug-interaction warnings do not appear

Probable causes: the expanded reference data was not seeded after install; two medications being tested are both present in the formulary but no interaction pair is seeded for them. **First diagnostic step**: `SELECT COUNT(*) FROM interaction_pairs;` Expect a non-zero result; if zero, re-run `seed_expanded.py` against the database.

Insurance claims stall in IN_REVIEW

Probable causes: the Pharmacist or Admin role has not processed them; an automation script (if one exists) has stopped; the InsuranceProvider record is missing configuration. **First diagnostic step**: query the claims table for claims older than 10 days in IN_REVIEW (the query is in Chapter 18) and discuss with the Clinical Administrator.

CHAPTER 28

Symptom Directory — Mobile and Desktop Clients

Client-side faults are disproportionately puzzling because the operator rarely has direct access to the client hardware and because the clients vary widely in their logging and diagnostic surface. The catalogue below lists the faults we have seen most often and the reproduction or workaround that resolves each.

iOS: login succeeds, prescriptions list is empty

Probable cause: the iOS client has cached an old JWT from a previous session while the JWT secret on the server has rotated. The URLSession silently accepts the server's 401, and the SwiftUI view renders its default empty state. **Fix**: instruct the user to log out and log back in. Consider adding a client-side check that treats an unexpected empty response following a successful login as a session error.

Android: "Trust anchor for certification path not found"

Probable cause: the server presents a certificate whose chain is incomplete, or a self-signed certificate whose root is not trusted by the device. **Fix**: ship the full chain on the server (Chapter 14) or, for internal deployments, install the internal CA's root on the device as a user CA. The `network_security_config.xml` bundled with the Android app permits user CAs on localhost and *10.0.2.2* for development; extend it only if you understand the security implications.

Windows WPF: app halts on login when server unreachable

This was a known defect in the 1.7.2 release (see commit c39fb0b) and has been fixed in 1.7.3. Clients running 1.7.2 that have lost connectivity to the server will hang at the login dialog. **Fix**: upgrade the Windows client to 1.7.3 or later. If immediate upgrade is infeasible, ensure that operators have a known-working server URL on the login screen before deploying the client.

macOS: SwiftUI Table column widths reset every launch

Probable cause: the user's preferences file is being reset by a macOS migration or a corrupted UserDefaults. **Fix**: this is cosmetic. If it is persistent, delete the `~/Library/Preferences/com.enlightec.medpharm.plist` file and relaunch; column widths will return to defaults, and subsequent customisations will persist.

All clients: Server URL has become invalid

Probable cause: an operator rehearsed a staging failover by pointing clients at the staging host and then forgot to point them back. Each client exposes a Server URL field with a "Reset to default" control; operators should instruct users to invoke Reset, which restores the default `https://medpharm-erp.enlightec.com:8080/api/v1` or the site's configured production endpoint.

Android: EncryptedSharedPreferences fails to open

Probable cause: the AndroidKeystore state has been corrupted — most often after a factory reset, a Google account migration, or a restore-from-backup onto a different device. **Fix**: clear the MedPharm app's data from system settings. This forces a fresh keystore registration on next launch. The user will need to log in again.

iOS: app refuses to launch, no visible error

Probable cause: a corrupted Keychain entry. **Fix**: uninstall and reinstall the app. Uninstallation clears the Keychain entries the app created; reinstallation begins fresh. The user will need to log in again.

Desktop (PyQt): QSslError on launch

The PyQt desktop does not normally make outbound HTTPS requests; if it reports a QSslError it is because an operator has configured it to talk to the remote API rather than the local database. **Fix**: if the desktop is meant to use the local database, ensure the connection string in `qt_app/settings.py` points at the local file and not the REST endpoint.

CHAPTER 29

Common Database Faults

SQLite has a quiet reputation, and deservedly so. It almost never misbehaves on its own. When it does, the cause is almost always something that the application or the operating environment has done to it. The faults in this chapter are the ones that actually occur; exotic problems are rare enough that when they happen the right response is to open a ticket with the SQLite maintainers.

Disk I/O error

SQLite reports *disk I/O error* when a call to `write()` returns an error from the underlying filesystem. On a Docker deployment this most often means the volume has filled; occasionally it means the host is experiencing hardware trouble. **First actions**: check free space with `df -h`; check kernel logs for I/O errors with `dmesg | tail`; if the disk is failing, failover to a standby host and replace the disk.

Database disk image is malformed

This is the most alarming error SQLite emits. It means the file's B-tree structure has been corrupted. **First action**: `sqlite3 medpharm_erp.db "PRAGMA integrity_check"` . If the check reports specific problems, attempt a dump-and-reload:

```
sqlite3 medpharm_erp.db ".dump" > recovery.sql
sqlite3 medpharm_erp.db.new < recovery.sql
# Diff the new file's row counts against expectations
sqlite3 medpharm_erp.db.new "SELECT COUNT(*) FROM patients;"
mv medpharm_erp.db.new /data/medpharm_erp.db
```

Danger. Do not attempt dump-and-reload on the live database while the application is running. Stop the application, take a filesystem copy of the damaged file (for forensics), and perform the recovery on the copy. If the recovery does not succeed, restore from backup (Chapter 20).

Database is locked

Already discussed in Chapter 27. Here the diagnostic deepens: after identifying the holding process, use `pragma busy_timeout = 5000` in application code if the contention is benign and short-lived, or restructure the offending query if the contention is long-lived.

WAL file grows without bound

The WAL file grows as long as at least one reader holds an old snapshot. If an application leaves a read transaction open indefinitely — for instance, a custom reporting script that uses a lazy iterator — the WAL cannot be checkpointed, and it grows. **First action**: identify the offending reader, close it, then run `PRAGMA`

`wal_checkpoint(TRUNCATE)`.

Unexpected row counts after restore

Occasionally a restore appears to succeed but the row counts do not match expectations. This is most often because the backup was taken at a moment inconsistent with the sidecar files — for example, a filesystem snapshot that captured the main database file without the WAL. **First action**: verify that the backup was produced via `.backup` or via a consistent snapshot mechanism, not by a raw file copy. If the raw copy is the only available restore point, open it with `sqlite3 backup.db PRAGMA journal_mode` — an error here indicates the sidecars are missing and the file has not fully checkpointed.

CHAPTER 30

Common Network Faults

Network faults are the category most often misattributed to the application. A patient whose cellular connection drops mid-request experiences the symptom as "the app didn't work," and the operator's first hour of investigation often vindicates the application entirely. Distinguishing application faults from network faults is therefore one of the most productive early acts of triage.

Triage by blast radius

A fault reported by a single user, from a single device, on a single network, is overwhelmingly likely to be a client or network fault. A fault reported by several users across several networks at the same time is almost certainly a server or upstream fault. A fault reported by several users but only on a given carrier's network is an upstream-routing fault. Ask the reporter two questions — "are you on Wi-Fi or cellular?" and "has anyone else in the practice said the same?" — before opening any terminal.

Typical symptoms and probable causes

Symptom	Probable Cause	First Diagnostic
Clients can't reach server	DNS change, firewall rule drift	dig / nslookup from several vantage points
Slow first connection, fast subsequent	TLS handshake latency; session-resumption disabled	openssl s_client -reconnect
Occasional TLS errors on mobile	Incomplete certificate chain	ssllabs.com/ssltest (or equivalent)
Dropped connections under load	Connection limits on Nginx worker	Nginx error log / worker_connections
Long tail latency on some users	Cross-region routing	traceroute from affected network

Proving the network is the problem

A short reproduction from a known-good vantage point is almost always decisive. If a curl from the operator's laptop succeeds, and the user's request fails, the network between the user and the server is the remaining variable. Run the curl with `-v` to capture timing and certificate information; save the output. If the operator's curl also fails, the problem is at or past the server's edge and the investigation shifts to Chapter 27.

> **Note.** Mobile carrier networks occasionally interpose captive portals, content-filtering proxies, or TLS inspection equipment. These are indistinguishable from a hostile man-in-the-middle to the client; MedPharm's mobile apps correctly refuse to connect through them, which is the intended behaviour. The usual mitigation is to instruct the user to switch to a different network.

PART VIII

MedPharm ERP Service Manual

Business Continuity

What happens when failure is larger than a single process and persists longer than a single hour.

CHAPTER 31

Failure Modes and Recovery Objectives

No system survives every conceivable failure; the best a serious deployment can do is survive the plausible ones and recover gracefully from the rest. This chapter enumerates the failure modes we treat as plausible for a MedPharm deployment, the detection signals we rely on for each, and the recovery objectives we believe are reasonable. The practice's own Disaster Recovery Plan should ratify or refine these numbers; they are a defensible starting point, not a commandment.

Catalogue of failure modes

Mode	Typical Cause	Detection	RPO	RTO
Process crash	Unhandled exception; OOM kill	Supervisor restart log; uptime monitor	None	< 5 min
Host failure	VM host down; network partition	Uptime monitor; pager	None (sync writes)	< 1 hr
Disk failure	Hardware; corruption	I/O errors in logs; integrity_check fail	≤ 24 hr	< 2 hr
Data corruption	Bug; operator error	Consistency check; user report	≤ 24 hr	< 4 hr
Ransomware	Compromise of host	Encryption of files; ransom note	≤ 24 hr	< 8 hr (depending on scope)
Regional outage	Cloud provider failure; data-centre event	Absence of uptime pings; provider status	≤ 24 hr	< 24 hr
Protracted outage	Sustained network; physical damage	Staff report; physical inspection	≤ 24 hr	Indeterminate; emergency mode

Legend. RPO is Recovery-Point Objective — the maximum tolerable data loss, measured in time. RTO is Recovery-Time Objective — the maximum tolerable outage duration.

The tolerability threshold

Not every failure justifies a DR activation. An API process crash that restarts within thirty seconds is noise, not an incident. A host failure that persists beyond ten minutes is an incident; beyond an hour, it is a DR activation. The threshold should be written down, known to all operators, and reviewed annually; the point is that when an operator is deciding whether to activate DR at 03:00, the decision is already made, merely to be read.

Dependencies on services we do not control

A MedPharm deployment typically depends, for its continued life, on three external services: a DNS provider, a TLS certificate issuer (unless an internal CA is used), and a backup storage provider. Failure in any of these shows up as MedPharm appearing to be down from a user's perspective. The operator should maintain contact information and SLA references for each, and should verify annually that the chosen providers' status pages reach the correct pagers.

CHAPTER 32

Disaster Recovery Runbooks

A runbook is a script for how to recover from a particular failure mode. It is written to be read in sequence and followed literally; deviations are allowed but must be recorded at the point of deviation so that the runbook may be improved afterwards. The runbooks in this chapter cover the three most common DR activations. Each is written in the imperative, present tense.

Runbook DR-1: Complete host loss

Trigger. The host on which MedPharm runs is no longer reachable and is not recoverable in under one hour. Typical causes: cloud host stopped by provider, hardware failure, accidental deletion of the VM.

1. Confirm the host is unreachable from at least two vantage points.
2. Verify the most recent backup is available at the off-site backup target.
3. Provision a replacement host of the same profile.
4. Install Docker and pull the MedPharm image.
5. Fetch, decrypt, and decompress the most recent backup onto the replacement host.
6. Place the backup at the data volume location (`/var/lib/docker/volumes/medpharm-data/_data/medpharm_erp.db`).
7. Install the TLS certificate (either by fetching the existing one from the certificate store, or by requesting a replacement from the CA).
8. Bring up the container and observe the startup banner.
9. Update DNS to point at the replacement host's IP address; confirm propagation from multiple resolvers.
10. Run the Chapter 7 validation matrix.
11. Announce restoration to clinical staff.
12. File the runbook deviation log (if any) and open a post-incident review.

> **Caution.** DNS propagation can take the better part of an hour depending on the TTL configured on the record. For deployments with aggressive RTO requirements, consider keeping TTLs low (300 seconds) as a matter of standing policy, so that the DNS step of DR-1 does not dominate the wall clock.

Runbook DR-2: Database corruption

Trigger. `PRAGMA integrity_check` reports damage; or a significant fraction of routine queries begin returning errors; or users report systematically missing data.

1. Stop the MedPharm container (graceful shutdown).

2. Take a filesystem copy of the damaged database file for forensic purposes. Name it `medpharm_erp.db.damaged-YYYYMMDD`.
3. Attempt dump-and-reload (Chapter 29). Preserve the recovered file as a candidate.
4. If the dump-and-reload produces a file whose row counts are within 1% of the most recent backup's row counts, prefer the dump-and-reload (more recent data).
5. Otherwise, restore from the most recent backup (Chapter 20).
6. Either way, run integrity_check against the restored database before starting the application.
7. Start the container; run the validation matrix.
8. Compare visible patient counts and prescription counts against end-of-day summaries from the past week; communicate any discrepancies to the Clinical Administrator immediately.

Runbook DR-3: Suspected compromise

Trigger. AuditLog anomalies; unexplained account creation; presence of files the operator did not write; ransom message. Treat as SEV-1 per Chapter 17.

1. Declare a SEV-1 incident; page the Security and Privacy Officers.
2. Firewall off the host from the internet at the cloud provider's security-group layer. Do not power the host off yet — volatile memory may be forensically useful.
3. Snapshot the host at the hypervisor layer.
4. Rotate MEDPHARM_JWT_SECRET and MEDPHARM_SECRET_KEY — even if the compromise is merely suspected.
5. Bring up a clean replacement host using the most recent backup that predates the earliest evidence of compromise.
6. Do not reuse any cryptographic material from the compromised host.
7. Retain the snapshotted original for as long as the Privacy Officer advises; it is part of the evidentiary record.
8. Conduct the post-incident review, consulting counsel regarding breach-notification obligations.

> **Danger.** Do not connect the compromised host to any production network, ever, after the incident. Adversaries who establish persistence often leave secondary implants that evade cursory inspection. The safe path is to treat the compromised host as irrecoverably untrusted and destroy it once forensics are complete.

CHAPTER 33

Business Continuity Planning

Business continuity is the wider concept that contains disaster recovery. DR asks how the system comes back; BCP asks how the practice continues to function until it does. For a small clinical organisation, BCP is less about complex automation and more about rehearsed manual workflows. A pharmacy whose ERP is unreachable does not stop dispensing medicine; it switches to a paper mode that has been planned in advance.

The "Emergency Mode" workflow

HIPAA § 164.308(a)(7) requires covered entities to have an Emergency Mode Operation Plan — a documented, rehearsed method of continuing critical business operations for the protection of ePHI during and immediately after an emergency. For MedPharm deployments this usually takes the form of a paper-and-phone workflow, engaged when the software is unavailable, and reconciled back into the database once service resumes.

- **Dispense from paper.** Controlled substances are prescribed on paper prescriptions written by the physician on duty. The paper log is entered into MedPharm within 48 hours of service restoration.
- **Record vital signs on paper.** Standard vital-signs forms are kept in each exam room and used for the duration of the outage.
- **Defer non-urgent scheduling.** Appointments scheduled during the outage are written on a single roster sheet at the front desk; this sheet is re-entered after restoration.
- **Defer billing.** Invoices are not written during the outage; visits are recorded on a single ledger and invoiced post-restoration.
- **Record drug-drug concerns manually.** Pharmacists cross-check new prescriptions against current regimens by consulting the printed formulary; automated interaction warnings are suspended.

Restoration reconciliation

When service resumes, the paper records produced during emergency mode must be re-entered into MedPharm promptly. This is work the clinical administrator plans for — typically a temporary data-entry resource is engaged — and the operator supports it by ensuring that entries made after restoration are timestamped correctly (with the original service date, not the date of entry). The AuditLog records both the original date and the entry date, so the practice has a defensible record of what happened when, and of what was entered when.

Annual continuity exercise

A continuity exercise is conducted annually. It does not need to be elaborate. A half-day simulated outage, during which clinical staff follow the emergency-mode workflow while the operator follows the DR runbooks, is sufficient to surface the procedural gaps that accumulate over a year. Each exercise produces a short written report that names the gaps and assigns their closure to someone; the following year's exercise re-tests those closures. A practice that conducts this exercise competently for three consecutive years is a practice in which a real incident will be, if not comfortable, at least not chaotic.

Note. The greatest contribution to business continuity is made not by the operator but by the practice's culture. A practice that treats outages as learning opportunities will, over time, become extremely resilient; one that treats them as blame opportunities will not. Operators should participate in shaping this culture, because the alternative is to re-encounter the same procedural failures at every exercise.

PART IX

MedPharm ERP Service Manual

Appendices

Reference material: commands, variables, ports, layout, glossary, and revision integrity.

APPENDIX A

Command Reference

This appendix collates the shell commands that appear most frequently throughout the manual. Commands are grouped by theme, and each is annotated with the chapter in which it is used. Commands expected to be run inside the MedPharm container are prefixed `docker exec medpharm-server …`; operators may wish to create a shell alias to shorten this.

Service lifecycle

Command	Purpose	Ref.
./install.sh	Commission MedPharm from source	Ch. 6
./install.sh --docker	Commission API-only Docker deployment	Ch. 6
./install.sh --docker-server	Commission full-stack Docker deployment	Ch. 6
docker compose up -d	Start services in the background	Ch. 9
docker compose stop	Graceful stop	Ch. 9
docker compose down	Stop and remove containers	Ch. 9
docker compose restart api	Restart just the API worker	Ch. 15
./uninstall.sh	Reverse the installation	Ch. 6
./uninstall.sh --all --force	Scorched-earth teardown	Ch. 6

Health and introspection

Command	Purpose	Ref.
curl -sv https://HOST/api/v1/health	Probe the REST health endpoint	Ch. 7
docker ps	List running containers	Ch. 8
docker logs --tail=200 medpharm-server	Tail recent container output	Ch. 27
docker exec medpharm-server supervisorctl status	Enumerate managed processes	Ch. 27
df -h /data	Check database volume free space	Ch. 8
openssl s_client -connect HOST:443 -servername HOST	Inspect the TLS certificate served	Ch. 14
sqlite3 medpharm_erp.db "PRAGMA integrity_check"	Verify database integrity	Ch. 18

Backup and restore

Command	Purpose	Ref.
/usr/local/bin/medpharm-backup.sh	Nightly backup script	Ch. 19
sqlite3 SRC ".backup 'DST'"	Hot snapshot a SQLite database	Ch. 19
gpg --decrypt foo.db.gz.gpg > foo.db.gz	Decrypt a GPG-encrypted backup	Ch. 20
gunzip foo.db.gz	Decompress a backup	Ch. 20
aws s3 cp FILE s3://BUCKET/	Ship backup to S3-compatible storage	Ch. 19

Security operations

Command	Purpose	Ref.
openssl rand -base64 48	Generate a 384-bit symmetric key	Ch. 6, 15
docker exec medpharm-server supervisorctl signal HUP nginx	Reload Nginx configuration	Ch. 14
openssl x509 -noout -enddate -in fullchain.pem	Print certificate expiry	Ch. 14
openssl x509 -noout -text -in fullchain.pem | grep DNS:	Print certificate SAN entries	Ch. 14

APPENDIX B

Environment Variable Reference

Every MedPharm deployment is configured by environment variables. This appendix is the authoritative list of those variables, their defaults, and their purpose. Unspecified variables take their defaults. Specified variables override the defaults regardless of where they originate — in the shell, in a compose file, or in a Kubernetes Secret. Operators should note that environment variables are not strongly typed; a misspelt name is silently ignored.

Variable	Default	Purpose
MEDPHARM_DB_PATH	medpharm_erp.db	Path to SQLite database file
MEDPHARM_SECRET_KEY	(auto-generated)	Flask session signing key
MEDPHARM_JWT_SECRET	(dev default)	HMAC-SHA256 key for JWTs
MEDPHARM_TOKEN_EXPIRY	86400	Access-token lifetime (seconds)
MEDPHARM_REFRESH_EXPIRY	604800	Refresh-token lifetime (seconds)
MEDPHARM_CORS_ORIGINS	*	Comma-separated allowed CORS origins
MEDPHARM_HOST	0.0.0.0	Listen interface
MEDPHARM_PORT	8080	Listen port (API)
MEDPHARM_DEBUG	false	Enable Flask debug mode (never in prod)
MEDPHARM_HTTP_PORT	80	Nginx HTTP listen (full stack)
MEDPHARM_HTTPS_PORT	443	Nginx HTTPS listen (full stack)
MEDPHARM_API_PORT	8080	Gunicorn port for API (full stack)
MEDPHARM_WEB_PORT	5000	Gunicorn port for portal (full stack)
MEDPHARM_WORKERS	4	Gunicorn worker processes
MEDPHARM_THREADS	2	Gunicorn threads per worker
MEDPHARM_TLS_MODE	auto	auto | require | disable
MEDPHARM_TLS_DIR	/etc/ssl/medpharm	Certificate directory
MEDPHARM_TLS_HOSTNAME	localhost	Hostname for self-signed cert
MEDPHARM_TLS_DAYS	825	Validity for generated self-signed cert
MEDPHARM_LOG_FORMAT	text	text | json
MEDPHARM_LOG_LEVEL	INFO	Application log level

Caution. `MEDPHARM_DEBUG=true` in a production deployment exposes the Werkzeug debugger, which is a remote code execution primitive under some circumstances. This must never be set in production, not even briefly, not even during incident response.

APPENDIX C

Port and Protocol Reference

A concise picture of which ports are expected to be open, in which direction, on a typical MedPharm deployment. Operators configuring firewalls should open only the ports required for the chosen topology; extraneous exposure is itself a compliance failure.

Inbound exposure

Port	Protocol	Source	Purpose
443/tcp	HTTPS	Internet (full stack)	Patient and staff traffic
80/tcp	HTTP	Internet (full stack)	301 redirect to 443
8080/tcp	HTTPS (direct)	Trusted network only	Gunicorn TLS for API (optional)
5000/tcp	HTTP (direct)	Localhost only	Gunicorn for portal (debug / loopback)
22/tcp	SSH	Bastion only	Operator access

Outbound dependencies

Target	Port	Purpose
Backup target (e.g. S3)	443/tcp	Off-host backup upload
Time source (NTP pool)	123/udp	Clock synchronisation
TLS issuer ACME endpoint	443/tcp	Let's Encrypt or equivalent
Upstream package mirrors	443/tcp	apt / pip / docker pull
Log aggregator	443/tcp	Structured log shipping
Monitoring endpoint	443/tcp	Heartbeats, uptime pings

Internal (container → container)

From	To	Port	Purpose
Nginx	Gunicorn (API)	8080/tcp	Proxy HTTP
Nginx	Gunicorn (Web)	5000/tcp	Proxy HTTP
Gunicorn	SQLite (on disk)	n/a	File I/O
Supervisor	All managed processes	n/a	Lifecycle

APPENDIX D

File and Directory Layout

The operator's view of the filesystem inside a running MedPharm container and on its host-side volumes. Paths marked with ● are operator-writable; those with ⊗ are deployment-controlled and should not be edited in place.

Path	Owner	Purpose
/data/medpharm_erp.db ●	app user	Live SQLite database
/data/medpharm_erp.db-wal	app user	Write-ahead log sidecar
/data/medpharm_erp.db-shm	app user	Shared-memory sidecar
/etc/ssl/medpharm/fullchain.pem ●	root	Server certificate chain
/etc/ssl/medpharm/privkey.pem ●	root	Server private key (mode 600)
/var/log/medpharm/ ●	root	Application and system logs
/var/backups/medpharm/ ●	root	On-host backup staging
/opt/medpharm/ ⊗	root	Application code (do not edit in place)
/opt/medpharm/api/ ⊗	root	Flask application factory
/opt/medpharm/database/ ⊗	root	SQLAlchemy models and seed data
/opt/medpharm/web/ ⊗	root	Flask patient portal
/opt/medpharm/qt_app/ ⊗	root	PyQt desktop (only in source installs)
/etc/nginx/conf.d/medpharm.conf ⊗	root	Nginx reverse-proxy config
/etc/supervisor/conf.d/medpharm.conf ⊗	root	Supervisor process manifest

APPENDIX E

Glossary of Terms

Brief definitions of the terms used throughout this manual. Where a term has both a general and a MedPharm-specific meaning, the MedPharm-specific is given.

Term	Definition
ACME	Automated Certificate Management Environment; the protocol Let's Encrypt uses to issue and renew certificates.
Admin role	Superuser role within the MedPharm application. Must be few and individually attributed.
API	Application Programming Interface. In MedPharm, the Flask-based REST endpoints under /api/v1/.
Audit log	The AuditLog database table; append-only record of sensitive actions.
BAA	Business Associate Agreement. HIPAA contract required with every third party handling PHI.
BCP	Business Continuity Plan. Procedures the practice follows during an outage.
CORS	Cross-Origin Resource Sharing. HTTP mechanism by which servers declare which origins may invoke them.
DPAPI	Data Protection API; Windows mechanism for user-scoped encryption at rest.
DR	Disaster Recovery. Procedures the practice follows to restore the system.
ePHI	Electronic Protected Health Information. Health data in electronic form, subject to HIPAA.
Gunicorn	Python WSGI server used to host the Flask application.
HIPAA	Health Insurance Portability and Accountability Act (1996). The principal U.S. law governing PHI handling.
JWT	JSON Web Token. The bearer-token format used by MedPharm clients for authentication.
Keychain	Apple's secure credential store; used by the iOS and macOS clients for JWT storage.
ORM	Object-Relational Mapper. In MedPharm, SQLAlchemy 2.0.
PBKDF2	Password-Based Key Derivation Function 2. Used by MedPharm for password hashing with SHA-256.
PHI	Protected Health Information. Patient-identifiable health data.
Post-mortem	Written review of an incident, its timeline, and its lessons. Blameless by convention.
RPO	Recovery Point Objective. The maximum tolerable data loss.
RTO	Recovery Time Objective. The maximum tolerable outage duration.
SAN	Subject Alternative Name. Field on an X.509 certificate listing the hostnames it covers.
Security Rule	HIPAA subpart C. Establishes technical, administrative, and physical safeguards for ePHI.
SEV-1 / SEV-2	Severity levels for incident classification; SEV-1 is most severe.

Term	Definition
Supervisor	Process manager running inside the MedPharm container; launches Nginx, Gunicorn, and sidecars.
WAL	Write-Ahead Log. SQLite journaling mode under which readers do not block writers and vice versa.

APPENDIX F

Revision History and Document Integrity

This manual is a living document. Revisions are numbered against the MedPharm release with which they are current, followed by a letter designating the revision of the manual itself within that release (e.g. 1.7.5-A, 1.7.5-B). A revision-history table must accompany the manual; operators are asked to add to it when they contribute changes.

Revision	Date	Author	Summary of Change
1.7.5-A	25 Apr 2026	R. Stillwell	Initial issue of the operator-facing service manual.

Integrity check

To ensure the manual's integrity during distribution, each authoritative revision is hashed and the hash published alongside the PDF on the MedPharm release page. Operators downloading the manual may verify that their copy matches the authoritative version.

```
sha256sum MedPharm_ERP_Service_Manual.pdf
# Compare the resulting hash against the value on the release page.
# Mismatch means the file has been altered or incompletely downloaded.
```

Contributions

Operators who identify errors or gaps are invited to submit corrections as pull requests against the MedPharm source repository. Changes to the narrative should preserve the manual's register and signal-word conventions; additions that introduce a new procedure should include a verification step and should be cross-referenced from any related chapters. Substantive revisions are announced in the MedPharm release notes.

Afterword

You have reached the end of the service manual. The document you have read is the accumulated experience of running MedPharm in real clinical settings, distilled into forms that we hope are usable under the three modes described in the front matter: cover-to-cover, as a reference, and under pressure. Most of the manual will, on any given day, be irrelevant to the operator's immediate work; what matters is that on the one day it becomes relevant, it reads clearly.

We close with a small request. Treat this manual as a working tool, not as a commemorative volume. Mark it up. Annotate it with what you have learned that it does not yet contain. And when those annotations accumulate, contribute them back, so that the next operator finds a document that is one increment better

than the one you inherited. That is the mechanism by which operational wisdom — the most valuable form of wisdom in clinical IT — survives the people who earned it.

End of the MedPharm ERP Service Manual, Volume II, Revision 1.7.5-A.